



Alvio

Plateforme de Trading propulsée par IA

Rapport de Projet — Titre Professionnel CDA

RNCP 37873 — Niveau 6 (Bac+4)

Adam Ijjai

Concepteur Développeur d'Applications

Promotion 2025-2026

NestJS 10 · Next.js 13 · React Native · Python IA · Claude API · PostgreSQL · Redis · Docker



Alvio

Plateforme de Trading propulsée par IA

Rapport de Projet — Titre Professionnel CDA

RNCP 37873 — Niveau 6 (Bac+4)

Adam Ijjai

Concepteur Développeur d'Applications

Promotion 2025-2026

NestJS 10 · Next.js 13 · React Native · Python IA · Claude API · PostgreSQL · Redis · Docker

Sommaire

Sommaire	3
1. Présentation & Remerciements	8
À propos du projet	8
Présentation du candidat	8
Périmètre du projet réalisé	8
Remerciements	9
2. À propos de moi	10
Profil	10
Compétences techniques mobilisées dans ce projet	10
Choix pédagogiques	11
3. Introduction	12
Contexte	12
Problématique	12
Livrables du projet	12
Périmètre fonctionnel — vue d'ensemble	14
Structure de ce dossier	14
4. Gestion de projet	16
Contexte organisationnel	16
Méthodologie — Scrum adapté au solo	16
Product Breakdown Structure (PBS)	16
Organisation du backlog — sprints réalisés	17

Gestion des risques	18
Outils de suivi	18
Bilan organisationnel	19
5. Conception fonctionnelle UI/UX	20
Charte graphique Alvio	20
Application Web — Zoning et pages réelles	20
Application Mobile — 5 écrans réels	22
Accessibilité & SEO	23
6. Conception de la Base de Données	25
Système de gestion de données	25
Dictionnaire de données	25
Modèle Conceptuel de Données (MCD)	31
Modèle Logique de Données (MLD)	32
Modèle Physique de Données (MPD)	33
7. Modélisation UML	35
Diagramme de cas d'utilisation	35
Diagramme de classes	36
Diagramme de séquence — Création de stratégie IA	39
Diagramme de séquence — Authentification multi-facteurs	41
8. Architecture Logicielle	43
Vue d'ensemble	43
Vue des endpoints (routes réelles)	44
Sécurité & sécurisation des endpoints	46

Gestion des vulnérabilités	48
Gestion des dépendances et configuration	49
Workflow : parcours complet d'une requête	50

9. Application Web 53

Vue d'ensemble	53
Authentification — flux côté client	54
Module Stratégies IA — interface d'import PDF	55
Page Marchés — données CoinGecko en temps réel	56
Module Formation (LMS intégré)	57
Sécurité en arrière-plan	58

10. Application Mobile 60

Vue d'ensemble	60
Navigation — Structure réelle	60
Les 5 écrans réels	61
Intégration backend	66
Perspectives d'évolution	67

11. Module IA / ML 69

Pipeline IA global	69
Import et parsing PDF	70
Strategy Engine IA — Appel Claude	71
Bridge Python — `spawn` stdin/stdout	72
Détection de patterns — module Python	75
Génération des signaux — design long-only	76
Rapports mensuels automatisés	78

Limites et perspectives d'évolution	79
12. Tests — Unitaires & Intégration	81
Stratégie de test	81
Tests unitaires Backend (Jest — 5 fichiers)	81
Tests composants Frontend (Vitest + RTL — 1 fichier)	86
Tests Python (`test_ai_module.py`)	86
État de la couverture — bilan honnête	88
Perspectives d'évolution des tests	88
13. Déploiement	90
Objectifs	90
Conteneurisation	90
CI/CD — État réel	93
Cibles de déploiement — Statut	93
Variables d'environnement de production	94
Perspectives CI/CD	95
14. Annexes	97
A. Diagrammes	97
B. Extraits de code complets	98
15. Mapping des Blocs de Compétences RNCP 37873 — CDA	102
Bloc 1 — Développer une application sécurisée	102
Bloc 2 — Concevoir et développer une application sécurisée organisée en couches	104
Bloc 3 — Préparer le déploiement d'une application sécurisée	105
Synthèse par bloc	107

Points forts de la démonstration107

1. Présentation & Remerciements

À propos du projet

Alvio est une plateforme de trading propulsée par intelligence artificielle, développée dans le cadre du Titre Professionnel **Concepteur Développeur d'Applications** (RNCP 37873, Niveau 6 — Bac+4), promotion 2025-2026.

Le projet couvre l'ensemble de la chaîne : application web, application mobile, API REST, module IA en Python et infrastructure de déploiement conteneurisée.

Présentation du candidat

Nom	Adam Ijjai
Formation	CDA — Concepteur Développeur d'Applications
Référentiel	RNCP 37873 — Niveau 6 (European Qualifications Framework)
Promotion	2025-2026
Type de projet	Projet personnel solo — conception, développement et déploiement intégraux

Parcours et motivations

Mon intérêt pour le trading algorithmique et l'IA appliquée aux marchés financiers est à l'origine d'Alvio. L'objectif était de construire un outil réel et utilisable : un moteur capable d'apprendre les stratégies de trading à partir d'un simple document PDF, puis de surveiller les marchés en continu et d'émettre des signaux d'entrée et de sortie fondés sur une analyse technique Python.

Ce projet m'a permis de mobiliser simultanément les compétences des trois blocs RNCP : développement sécurisé (Bloc 1), conception en couches (Bloc 2) et déploiement (Bloc 3), en les appliquant à un domaine métier concret et exigeant.

Périmètre du projet réalisé

Le tableau ci-dessous résume les quatre périmètres techniques couverts :

Périmètre	Technologie principale	Statut
API REST backend	NestJS 10 · TypeScript · Prisma v5 · PostgreSQL 15	Réalisé
Application web	Next.js 13 · Zustand · Axios · Tailwind CSS	Réalisé
Application mobile	React Native 0.72.3 · Expo 49 · 5 écrans	Réalisé

Périmètre	Technologie principale	Statut
Module IA / Python	15 scripts Python · pandas · TA-Lib · scikit-learn · Claude API	Réalisé
Infrastructure	Docker 7 services · Prometheus · Grafana · ELK Stack	Réalisé

Remerciements

Je souhaite adresser mes sincères remerciements :

- **À l'équipe pédagogique** de ma formation CDA, pour l'encadrement technique et les retours critiques tout au long du projet.
- **À la communauté open source** — NestJS, Prisma, React Native, matplotlib — dont les outils et la documentation ont permis d'avancer efficacement dans un projet aussi transversal.
- **À Anthropic**, dont le modèle **claude-sonnet-4-6** est au cœur du Strategy Engine d'Alvio, et dont l'API publique a rendu possible l'intégration d'un LLM dans une architecture REST standard.
- **À CoinGecko**, dont l'API publique (v3) fournit l'intégralité des données de marché affichées dans Alvio.

*“Construire seul un projet complet — de la base de données à l'application mobile en passant par le module IA — est la meilleure façon de comprendre les interactions entre chaque couche d'une architecture logicielle moderne.”**

2. À propos de moi

Profil

Je suis développeur fullstack en formation **Concepteur Développeur d'Applications** (CDA — RNCP 37873, Niveau 6), avec une appétence marquée pour les architectures backend complexes, l'intelligence artificielle appliquée et la sécurité logicielle.

Alvio est mon projet de titre : je l'ai conçu, développé et déployé seul, de la modélisation de la base de données jusqu'à l'application mobile, en passant par un module IA Python intégré via bridge **spawn**.

Compétences techniques mobilisées dans ce projet

Backend & API

- **NestJS 10** (TypeScript) — architecture modulaire en couches (controllers, services, guards, strategies Passport)
- **Prisma v5** — ORM typé sur PostgreSQL 15, 11 modèles, migrations gérées
- **Redis 7** via **ioredis** — cache anti-429, gestion des sessions, compteurs de verrouillage
- **JWT** avec rotation JTI — access token 15 min en mémoire + refresh token 7 j en cookie httpOnly

Frontend & Mobile

- **Next.js 13** (Pages Router) avec **Zustand** pour le state et **Axios** pour les appels API (intercepteurs 401 / refresh automatique)
- **lightweight-charts 4.2.3** (TradingView) pour les graphiques OHLCV
- **React Native 0.72.3 / Expo 49** — application mobile avec navigation Stack + Bottom Tabs, 5 écrans fonctionnels

Intelligence artificielle

- **Claude API** (**claude-sonnet-4-6**) — analyse de documents PDF de stratégie et extraction de règles JSON structurées
- **Python** (pandas 2.0.3, numpy 1.24.3, TA-Lib 0.4.27, scikit-learn 1.3.0) — 15 scripts d'analyse technique, appelés via bridge **spawn** stdin/stdout depuis NestJS
- Indicateurs implémentés : RSI, MACD, Stochastique, Bandes de Bollinger, ATR, Ichimoku, ondes d'Elliott, patterns harmoniques (Gartley, Butterfly, Bat, Crab)

Sécurité & Authentification

- **Multi-facteurs (MFA)** : OTP par email, TOTP (RFC 6238, chiffré AES-256-GCM), WebAuthn FIDO2 (@simplewebauthn/server), PIN haché bcrypt
- Verrouillage de compte configurable (MAX_AUTH_FAILURES=3, LOCK_TTL_SECONDS=1800) géré dans Redis
- Audit trail complet dans la table AuthLog + export RFC 5424 Syslog UDP → Logstash

DevOps & Observabilité

- **Docker Compose** — 7 services orchestrés (postgres, redis, backend, frontend, prometheus, grafana, logstash)
- **Prometheus + Grafana** — 17 panneaux de métriques sécurité
- **ELK Stack** (Elasticsearch 8, Logstash, Kibana) — 5 règles SIEM de détection

Choix pédagogiques

Ce projet a délibérément évité les raccourcis :

- Pas de service d'authentification tiers (pas d'Auth0, pas de Firebase Auth) — l'ensemble du système MFA est implémenté from scratch pour maîtriser chaque mécanisme.
- Pas de ORM "magic" caché derrière une abstraction — Prisma est utilisé directement, avec des requêtes typées et des index optimisés.
- Le bridge Python n'utilise pas `child_process.exec` mais `spawn` avec `stdio: ['pipe', 'pipe', 'pipe']` pour une meilleure gestion des flux et de la mémoire.

Ces choix génèrent plus de complexité, mais ils correspondent précisément aux attendus d'un Concepteur Développeur d'Applications de niveau 6 : concevoir des architectures robustes, sécurisées, et maintenables.

3. Introduction

Contexte

Le trading de cryptomonnaies est aujourd'hui accessible à tous, mais les outils d'analyse technique restent l'apanage des traders expérimentés ou des équipes disposant de ressources importantes. Un trader individuel dispose rarement d'un système capable de surveiller les marchés en continu, de détecter des patterns complexes (formations chartistes, ondes d'Elliott, patterns harmoniques) et d'émettre des alertes exploitables en temps réel.

Par ailleurs, les Large Language Models (LLM) comme Claude ont rendu possible une nouvelle approche : **décrire une stratégie de trading en langage naturel ou dans un document PDF**, et laisser l'IA en extraire des règles formelles exploitables par un moteur d'exécution.

C'est ce double constat — manque d'outillage accessible + émergence des LLM — qui est à l'origine d'**Alvio**.

Problématique

Comment permettre à un trader individuel de formaliser, activer et surveiller sa propre stratégie de trading, en s'appuyant sur l'intelligence artificielle pour l'analyse et la détection de patterns, sans nécessiter de compétences en développement ou en data science ?

Cette problématique soulève trois défis techniques concrets :

- 1. Extraction de connaissance** — comment transformer un document de stratégie (PDF, texte) en règles machine exploitables de manière fiable ?
 - 2. Détection temps réel** — comment analyser en continu des données de marché (OHLCV) avec des algorithmes Python, intégrés dans une API REST NestJS sans bloquer le thread principal ?
 - 3. Sécurité et confiance** — un outil financier manipule des données sensibles et doit garantir une authentification robuste, un audit trail complet et une résistance aux attaques communes (CSRF, XSS, SQLi, brute-force).
-

Livrables du projet

1. API REST — `broker-ia-backend` (NestJS 10)

Backend principal exposant 14 controllers et une quarantaine d'endpoints REST. Héberge le Strategy Engine, le bridge Python, le planificateur de signaux et l'intégralité du système d'authentification MFA.

Caractéristique	Valeur
Framework	NestJS 10 + TypeScript
ORM	Prisma v5 — PostgreSQL 15
Cache	Redis 7 via ioredis
Port	3001

2. Application Web — `alvio-frontend` (Next.js 13)

Interface utilisateur complète : dashboard temps réel, page marchés avec graphiques chandeliers (lightweight-charts), gestion des stratégies et des signaux, module formation (LMS), simulateur DCA, rapports mensuels, et panneau de sécurité MFA.

Caractéristique	Valeur
Framework	Next.js 13.4.0 (Pages Router)
State	Zustand 4.5.7
Graphiques	lightweight-charts 4.2.3
Port	3000

3. Application Mobile — `broker-ia-mobile` (React Native)

Application mobile compagnon avec 5 écrans fonctionnels : authentification, dashboard, signaux, simulateur DCA et profil.

Caractéristique	Valeur
Framework	React Native 0.72.3
Toolchain	Expo 49
Navigation	Stack + Bottom Tabs (@react-navigation)

Note de périmètre : L'écran Markets (consultation des prix en temps réel)

n'est pas implémenté dans la version mobile actuelle — il constitue une perspective d'évolution naturelle.

4. Module IA Python (`ai-module`)

15 scripts d'analyse technique appelés depuis NestJS via bridge `spawn` stdin/stdout. Détecte les patterns chartistes et calcule les indicateurs (RSI, MACD, Bollinger, ATR, Stochastique, Ichimoku, ondes d'Elliott, patterns harmoniques).

Caractéristique	Valeur
Langage	Python 3
Libs principales	pandas 2.0.3 · numpy 1.24.3 · TA-Lib 0.4.27 · scikit-learn 1.3.0
Intégration	<code>spawn</code> stdin/stdout (pas <code>exec</code>)

Caractéristique	Valeur
Point d'entrée	pattern_detector.py

5. Infrastructure (`docker-compose.yml`)

7 services Docker orchestrés pour un déploiement reproductible en une commande.

Service	Image	Rôle
postgres	postgres:15	Base de données principale
redis	redis:7	Cache + sessions
backend	(build local)	API NestJS
frontend	(build local)	App Next.js
prometheus	prom/prometheus	Collecte métriques
grafana	grafana/grafana	Dashboard 17 panneaux
logstash	logstash:8	Pipeline logs → Elasticsearch

Périmètre fonctionnel — vue d'ensemble



Structure de ce dossier

Ce dossier de projet est organisé selon les blocs de compétences RNCP CDA :

Section	Bloc RNCP
Gestion de projet	Transversal
Conception UI/UX	Bloc 2 — Conception
Conception BDD (MCD/MLD/MPD)	Bloc 2 — Conception
Modélisation UML	Bloc 2 — Conception

Section	Bloc RNCP
Architecture logicielle	Bloc 2 — Conception
Application Web	Bloc 1 — Développement sécurisé
Application Mobile	Bloc 1 — Développement sécurisé
Module IA/ML	Bloc 1 — Développement sécurisé
Tests	Bloc 1 — Développement sécurisé
Déploiement	Bloc 3 — Déploiement
Sécurité	Bloc 1 — Développement sécurisé
Mapping RNCP	Transversal

4. Gestion de projet

Contexte organisationnel

Alvio est un **projet solo** : conception, développement, tests et déploiement sont assurés par une seule personne. La gestion de projet a donc été adaptée en conséquence — sans réunions d'équipe, sans matrice RACI multi-acteurs, mais avec une discipline personnelle rigoureuse sur le découpage en sprints, la gestion du backlog et le suivi des risques.

Méthodologie — Scrum adapté au solo

La méthode Scrum a été retenue pour sa structure itérative et sa capacité à produire des incréments fonctionnels à intervalles réguliers. Les cérémonies ont été allégées pour s'adapter au travail individuel :

Cérémonie Scrum standard	Adaptation solo
Sprint Planning (équipe)	Sélection personnelle du backlog en début de sprint
Daily Scrum (15 min, équipe)	Note quotidienne (bloc-notes) : fait hier / à faire / bloquants
Sprint Review (demo équipe)	Auto-review : test des fonctionnalités du sprint sur l'environnement local
Sprint Retrospective	Analyse des écarts (durée estimée vs réelle) et ajustement du prochain sprint
Product Owner	Rôle assumé par moi-même — arbitrage des priorités selon la valeur métier

Durée des sprints : 1 à 2 semaines selon la complexité du module traité.

Product Breakdown Structure (PBS)

Le PBS décompose le produit Alvio en composants livrables indépendants :

```
ALVIO
├── 1. Backend API (NestJS)
│   ├── 1.1 Module Auth (email/password, Magic Link, GitHub, OTP, TOTP, WebAuthn, PIN)
│   ├── 1.2 Module Marchés (CoinGecko, cache Redis, OHLCV)
│   ├── 1.3 Module Stratégies (upload PDF, pdf-parse v2, analyse Claude)
│   ├── 1.4 Module Signaux (génération, scheduler 15 min, long-only)
│   ├── 1.5 Module Formation (Course / Lesson / UserProgress)
│   ├── 1.6 Module Simulateur (DCA fixed + monte_carlo)
│   ├── 1.7 Module Rapports (stats mensuelles par userId/year/month)
│   └── 1.8 Module Monitoring (Prometheus, Logging RFC 5424, Métriques)
└── 2. Application Web (Next.js)
    ├── 2.1 Pages publiques (landing, login, register)
    ├── 2.2 Dashboard
    ├── 2.3 Pages Marchés (top 20 + graphique chandelier)
    ├── 2.4 Pages Stratégies (liste, import PDF, analyse)
    └── 2.5 Pages Signaux (liste, détail)
```

- ■■■ 2.6 Module Formation (liste cours, leçon, progression)
- ■■■ 2.7 Simulateur DCA
- ■■■ 2.8 Rapports mensuels
- ■■■ 2.9 Profil & Sécurité (MFA, 2FA, WebAuthn)
- ■■■ 3. Application Mobile (React Native / Expo)
- ■■■ 3.1 Écran Login
- ■■■ 3.2 Écran Dashboard
- ■■■ 3.3 Écran Signaux
- ■■■ 3.4 Écran Simulateur
- ■■■ 3.5 Écran Profil
- ■■■ 4. Module IA Python
- ■■■ 4.1 Détection patterns chartistes (candlestick, chart, harmoniques, Elliott)
- ■■■ 4.2 Calcul indicateurs (RSI, MACD, Bollinger, ATR, Stochastique, Ichimoku)
- ■■■ 4.3 Scoring engine (confidence_score 0-100)
- ■■■ 4.4 Bridge NestJS→Python (spawn stdin/stdout)
- ■■■ 4.5 Simulateur DCA Python (dca_simulator.py)
- ■■■ 5. Infrastructure
- ■■■ 5.1 Docker Compose (7 services)
- ■■■ 5.2 Prometheus + Grafana (17 panneaux)
- ■■■ 5.3 ELK Stack (5 règles SIEM Kibana)

Organisation du backlog — sprints réalisés

Sprint	Durée	Objectif principal	Livrables clés
S1	2 sem.	Fondations backend	Prisma schema (11 modèles), PrismaService, RedisService, AppModule
S2	2 sem.	Authentification complète	AuthService (519 L), JWT JTI, MFA email OTP, PIN, TOTP, WebAuthn
S3	1 sem.	Marchés & cache	MarketsService, CoinGecko, cache Redis anti-429, stale fallback
S4	2 sem.	Strategy Engine IA	pdf-parse v2, AIService Claude (claude-sonnet-4-6), StrategyRules
S5	2 sem.	Module Python & signaux	15 scripts Python, bridge spawn , SignalsService long-only, scheduler
S6	1 sem.	Formation & simulateur	FormationService, SimulatorService (Box-Muller), ReportsService
S7	1 sem.	Frontend web complet	20+ pages Next.js, Zustand, intercepteurs Axios, lightweight-charts
S8	1 sem.	Application mobile	5 écrans React Native / Expo, navigation Stack + Bottom Tabs

Sprint	Durée	Objectif principal	Livrables clés
S9	1 sem.	Monitoring & tests	Prometheus, Grafana, ELK, 5 spec Jest, Vitest, test_ai_module.py
S10	1 sem.	Docker & finalisation	docker-compose.yml (7 services), README, dossier jury

Gestion des risques

Risque	Probabilité	Impact	Mitigation appliquée
API CoinGecko → erreur 429 (rate limit)	Haute	Moyen	Cache Redis double-clé (TTL frais + TTL stale 1 h)
Claude API → réponse hors format JSON	Moyenne	Haut	<code>cleanJsonResponse()</code> + <code>validateStrategyRules()</code> + relance
TA-Lib introuvable sur la machine cible	Moyenne	Haut	Docker image avec TA-Lib compilé ; fallback numpy pur
Perte de données Redis (redémarrage)	Faible	Moyen	Sessions critiques persistées en base (AuthLog) ; Redis non persistant par choix
Dérive du périmètre (scope creep)	Haute (projet solo)	Moyen	Backlog figé après S4 ; features reportées en perspectives
Compilation TypeScript → erreurs Prisma	Réalisé	Haut	<code>npx prisma generate</code> intégré au <code>npm run build</code> ; types Prisma importés explicitement
Tests insuffisants sur les paths critiques	Réalisée	Haut	Jest seuil 70 % (branches, fonctions, lines, statements)

Outils de suivi

Outil	Usage dans ce projet
Git (mono-repo)	Versionning de l'ensemble du code (backend, frontend, mobile, ai-module)
Commits conventionnels	<code>feat:</code> , <code>fix:</code> , <code>refactor:</code> , <code>test:</code> — historique lisible
Bloc-notes personnel	Backlog, notes de sprint, suivi des bloquants
docker-compose	Reproductibilité de l'environnement de dev/test en une commande
Jest --coverage	Vérification du seuil de couverture à chaque sprint

Bilan organisationnel

Travailler seul sur un projet de cette envergure impose une discipline stricte sur deux points critiques :

1. Découper avant de coder — le PBS a permis d'identifier les dépendances entre modules (le Strategy Engine nécessite Redis opérationnel, le bridge Python nécessite les données OHLCV de MarketsService) et de les traiter dans le bon ordre.

2. Accepter de reporter — plusieurs fonctionnalités initialement prévues (`backtestStrategy`, `getPatternsHistory`, écran Markets mobile) ont été consciemment reportées en perspectives pour préserver la qualité des modules réalisés.

5. Conception fonctionnelle UI/UX

Charte graphique Alvio

La charte graphique d'Alvio est construite autour d'une identité visuelle **trading premium** : fond sombre (navy), accents chauds (amber) pour les données importantes, vert pour les signaux positifs et violet pour les fonctions secondaires.

Palette de couleurs

Rôle	Couleur	Code hex	Usage
Fond principal	Navy	#0f172a	Background global, cartes
Primaire / CTA	Amber	#f59e0b	Boutons, prix, valeurs clés, titres
Signaux positifs	Vert	#10b981	BUY, hausse, succès, confiance
Accent / secondaire	Violet	#6366f1	Badges, étapes MFA, navigation active
Texte secondaire	Slate	#94a3b8	Labels, descriptions, métadonnées
Surface carte	Glass	#1e293b	Cartes en glassmorphism

Style visuel

- **Glassmorphism** : cartes avec fond semi-transparent (#1e293b), bordure fine (1px solid rgba(255,255,255,0.08)) et backdrop-filter: blur(12px).
- **Typography** : Inter (ou DejaVu Sans en fallback) — hiérarchie en poids 400 / 600 / 700.
- **Animations** : transitions fluides via framer-motion 12.34.1 (fade-in, slide-up pour les modales et les notifications).
- **Logo** : triangle + ligne graphique ascendante — symbole d'une tendance haussière, cohérent avec le positionnement trading.
- **Icônes** : ionicons 7.2.1 (mobile) + SVG inline (web).

Application Web — Zoning et pages réelles

Structure de navigation (Next.js 13 Pages Router)

```
/ (landing)
├── /login      ← email/password → 2FA → PIN
├── /register
└── /dashboard ← vue d'ensemble (signaux récents, marchés top 3, stats)
```

Zoning — Dashboard (page principale connectée)

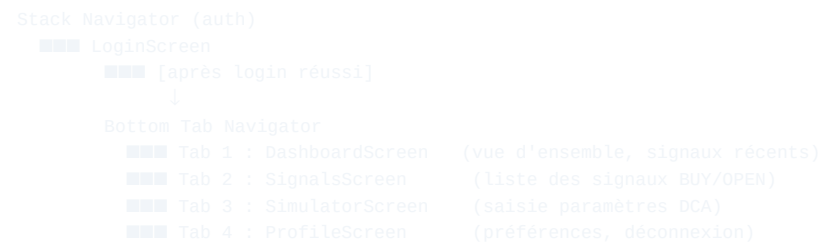
Zoning — Page Marchés `/markets/[id]`

Zoning — Import stratégie `/strategies/import`

```
=====
■  IMPORT DE STRATÉGIE                               ■
=====
■
■                                     ■
■  =====                                     ■
■  ■ [ Glisser un fichier PDF ou cliquer pour sélectionner ] ■ ■
■  ■ (PDF, TXT, MD — max 10 MB)                ■ ■
■  =====                                     ■
■
■                                     ■
■  Nom de la stratégie : [_____]                ■
■  Asset : [BTC ▼]   Timeframe : [1h ▼]           ■
■
■  [ Analyser avec Claude AI ] ← POST /strategies/import
■
■                                     ■
■  =====                                     ■
■  ■ RÉSULTAT — StrategyRules extraites :        ■ ■
■  ■ • entry_conditions : RSI < 30, EMA20 > EMA50      ■ ■
■  ■ • exit_conditions  : RSI > 70 OR stop_loss atteint  ■ ■
■  ■ • risk_management  : stop_loss 2%, take_profit 6%   ■ ■
■  ■ • confidence_score : 87                          ■ ■
■  ■
■  =====                                     ■
=====
```

Application Mobile — 5 écrans réels

La navigation mobile combine un **Stack Navigator** (auth) et un **Bottom Tab Navigator** (écrans connectés).



Note : l'écran Markets n'est pas implémenté dans la version mobile actuelle.

C'est une perspective d'évolution documentée dans la section Déploiement.

Zoning mobile — DashboardScreen

```
=====
■ ▲ ALVIO [Notif] ■ ← Header amber
=====
■
■                                     ■
■  Bonjour, Adam                      ■
■  =====                         ■
■  ■ BTC · BUY · OPEN ■ ← SignalCard (glassmorphism)
■  ■ conf. 87%        ■ ■
■  ■ entry $67,420    ■ ■
■  ■ =====         ■
■  ■ =====         ■
■  ■ ETH · BUY · OPEN ■
■  ■ conf. 72%        ■ ■
■  ■ =====         ■
■
■                                     ■
■  [Voir tous les signaux] ■
■  =====
■  [Dashboard][Signals]   ■ ← Bottom Tabs (icône ionique)
```

Zoning mobile — SimulatorScreen

Accessibilité & SEO

SEO (Next.js)

Next.js 13 (Pages Router) fournit le SSR et la génération statique nativement, ce qui favorise l'indexabilité :

- **Balises** `<title>` et `<meta description>` personnalisées par page via le composant `<Head>` de `next/head`.

- **Open Graph** : balises `og:title`, `og:description`, `og:image` pour le partage sur les réseaux sociaux.

- **URL sémantiques** : `/markets/bitcoin`, `/formation/courses/rsi-macd`.
- **Proxy API** (`/api/coingecko/*`) : les données de marché passent par Next.js pour éviter l'exposition de la clé API côté client et permettre le cache côté serveur.

Pages publiques indexables

Page	Contenu indexable
/ (landing)	Présentation Alvio, fonctionnalités, CTA
/pricing	Tarifs (page statique)

Les pages connectées (`/dashboard`, `/signals`, etc.) sont protégées côté serveur (`getServerSideProps` + vérification JWT) — elles ne sont pas indexées.

Accessibilité

- Contraste respecté entre le texte blanc (#f1f5f9) et le fond navy (#0f172a) — ratio > 7:1 (WCAG AA).
- Navigation clavier fonctionnelle sur les formulaires d'authentification.

- Labels `aria-label` sur les boutons d'icônes (graphiques, fermeture de modales).

6. Conception de la Base de Données

Système de gestion de données

Paramètre	Valeur
SGBD	PostgreSQL 15
ORM	Prisma v5.22.0
Nombre de modèles	11
Clés primaires	CUID (<code>@default(cuid())</code>) — identifiants courts et non-séquentiels
Environnement dev	SQLite possible (fichier <code>dev.db</code>)
Environnement prod	PostgreSQL 15 (Docker <code>postgres:15</code>)

Dictionnaire de données

USER — Utilisateur

Entité centrale du système. Supporte plusieurs modes d'authentification (email/password, GitHub OAuth, Magic Link) et plusieurs facteurs MFA (OTP, TOTP, WebAuthn, PIN).

Champ	Type SQL	Contraintes	Description
<code>id</code>	CHAR(25)	PK, NOT NULL	CUID généré automatiquement
<code>username</code>	VARCHAR	UNIQUE, NOT NULL	Identifiant affiché
<code>email</code>	VARCHAR	UNIQUE, NOT NULL	Adresse email principale
<code>passwordHash</code>	TEXT	NULL	Hash bcrypt 12 rounds ; NULL si auth GitHub/MagicLink
<code>githubId</code>	VARCHAR	UNIQUE, NULL	ID GitHub OAuth ; NULL si auth classique
<code>pin</code>	TEXT	NULL	3ème facteur ; hashé bcrypt
<code>totpSecret</code>	TEXT	NULL	Secret TOTP chiffré AES-256-GCM
<code>totpEnabled</code>	BOOL	DEFAULT false	TOTP activé/désactivé
<code>trading_preference</code>	VARCHAR	DEFAULT 'moderate'	Profil de risque (<code>conservative</code> / <code>moderate</code> / <code>aggressive</code>)
<code>email_notifications</code>	BOOL	DEFAULT true	Notifications email signaux
<code>createdAt</code>	TIMESTAMPTZ	DEFAULT now()	Date de création
<code>updatedAt</code>	TIMESTAMPTZ	@updatedAt	Mise à jour automatique

Relations : → `Signal[]`, `Strategy[]`, `Report[]`, `WebAuthnCredential[]`, `UserProgress[]`, `PortfolioSnapshot[]`, `SimulationResult[]`

SIGNAL — Signal de trading

Représente une opportunité d'entrée ou de sortie de marché détectée par le moteur IA. Design **long-only** : `direction` vaut toujours `BUY` pour une entrée ; la sortie est modélisée par `status='CLOSED'` + `exit_price` sur le signal `BUY` existant (pas de signal `SELL` distinct).

Champ	Type SQL	Contraintes	Description		
<code>id</code>	CHAR(25)	PK	CUID		
<code>userId</code>	CHAR(25)	FK → User, IDX	Propriétaire du signal		
<code>strategyId</code>	VARCHAR	NULL, IDX	Stratégie source (optionnel)		
<code>asset</code>	VARCHAR(20)	NOT NULL, IDX	Ex : <code>BTC</code> , <code>ETH</code> , <code>SOL</code>		
<code>timeframe</code>	VARCHAR	NULL	Ex : <code>15m</code> , <code>1h</code> , <code>4h</code> , <code>1d</code>		
<code>direction</code>	VARCHAR(10)	NOT NULL	<code>BUY</code> \	<code>SELL</code> \	<code>HOLD</code>
<code>status</code>	VARCHAR(10)	DEFAULT 'OPEN'	<code>OPEN</code> \	<code>CLOSED</code>	
<code>entry_price</code>	FLOAT8	NOT NULL	Prix d'entrée		
<code>stop_loss</code>	FLOAT8	NOT NULL	Niveau stop-loss		
<code>take_profit</code>	FLOAT8	NOT NULL	Objectif de profit		
<code>exit_price</code>	FLOAT8	NULL	Renseigné à la clôture (EXIT_SIGNAL)		
<code>confidence</code>	FLOAT8	NOT NULL	Score de confiance 0–100 (%)		
<code>risk_reward_ratio</code>	FLOAT8	NULL	Ratio risque/rendement		
<code>patterns</code>	TEXT	NULL	JSON array — patterns détectés		
<code>indicators</code>	TEXT	NULL	JSON object — valeurs indicateurs		
<code>closedAt</code>	TIMESTAMPTZ	NULL	Date de clôture		
<code>createdAt</code>	TIMESTAMPTZ	NOT NULL	Date de création		

Index composite : [`strategyId`, `asset`, `direction`, `status`] — utilisé par la déduplication (`findFirst` pour éviter deux `BUY OPEN` sur le même `asset`/stratégie).

STRATEGY — Stratégie de trading

Contient les règles JSON extraites par Claude à partir d'un document PDF. Le champ `code` stocke un objet `StrategyRules` sérialisé.

Champ	Type SQL	Contraintes	Description			
id	CHAR(25)	PK	CUID			
userId	CHAR(25)	FK → User, IDX	Propriétaire			
name	TEXT	NOT NULL	Nom de la stratégie			
description	TEXT	NULL	Description libre			
code	TEXT	NOT NULL	JSON StrategyRules (entry/exit conditions, risk_management)			
asset	VARCHAR(20)	NOT NULL	Actif ciblé			
timeframe	VARCHAR(5)	NOT NULL	15m \	1h \	4h \	1d
status	VARCHAR	DEFAULT 'inactive'	inactive \	active		
win_rate	FLOAT8	NULL	Taux de réussite (calculé)			
total_trades	INT	NULL	Nombre de trades			
profit_factor	FLOAT8	NULL	Facteur de profit			

REPORT — Rapport mensuel

Agrégation des statistiques de trading par mois, calculée par ReportsService à partir des signaux de l'utilisateur.

Champ	Type SQL	Contraintes	Description
id	CHAR(25)	PK	CUID
userId	CHAR(25)	FK → User	Propriétaire
month	SMALLINT	NOT NULL	Mois (1–12)
year	SMALLINT	NOT NULL	Année
total_signals	INT	NOT NULL	Entrées + sorties du mois
buy_signals	INT		Entrées (createdAt dans le mois)
sell_signals	INT		Sorties (closedAt dans le mois)
hold_signals	INT		Positions OPEN en fin de mois
win_rate	FLOAT8		% clôtures avec exit_price > entry_price
avg_confidence	FLOAT8		Confiance moyenne des entrées
best_signal_confidence	FLOAT8		Meilleure confiance du mois

Champ	Type SQL	Contraintes	Description
worst_signal_confidence	FLOAT8		Pire confiance du mois
total_pnl_estimate	FLOAT8		Somme (exit_price – entry_price)
total_trades_expected	INT		max(buy_signals, sell_signals)
high_confidence_signals	INT		Signaux avec confiance > 80 %
patterns_detected	TEXT	NULL	JSON — comptage par pattern
indicators_used	TEXT	NULL	JSON — liste des indicateurs
summary	TEXT	NULL	Texte résumé auto-généré

Contrainte unique : [userId, month, year]

AUTH_LOG — Journal d'audit d'authentification

Trace chaque tentative d'authentification. **userId** est nullable pour capturer les tentatives sur des emails inexistants.

Champ	Type SQL	Contraintes	Description		
id	CHAR(25)	PK	CUID		
userId	TEXT	NULL	Utilisateur ciblé (si connu)		
action	VARCHAR(50)	NOT NULL	login, register, 2fa_verify, pin_verify...		
result	VARCHAR(20)	NOT NULL	success \	failure \	blocked
ip	VARCHAR(45)	NULL	Adresse IP source (IPv4 ou IPv6)		
detail	TEXT	NULL	Message d'erreur ou contexte		
createdAt	TIMESTAMPTZ	DEFAULT now()	Timestamp		

WEBAUTHN_CREDENTIAL — Clé WebAuthn FIDO2

Une clé par appareil enregistré. Le **counter** est vérifié à chaque authentification pour prévenir les attaques par replay.

Champ	Type SQL	Contraintes	Description	
id	CHAR(25)	PK	CUID	
userId	CHAR(25)	FK → User, IDX	Propriétaire	
credentialId	TEXT	UNIQUE	Identifiant base64url de la clé	

Champ	Type SQL	Contraintes	Description	
publicKey	TEXT	NOT NULL	Clé publique COSE base64url	
counter	INT	DEFAULT 0	Compteur anti-rejeu	
deviceType	VARCHAR(20)	NULL	singleDevice \	multiDevice
backedUp	BOOL	DEFAULT false	Clé sauvegardée cloud	
transports	TEXT	NULL	JSON array (usb, nfc, internal...)	
aaguid	VARCHAR	NULL	Identifiant du modèle d'authenticateur	
lastUsedAt	TIMESTAMPZ	NULL	Dernière utilisation	

SIMULATION_RESULT — Résultat de simulation DCA

Persistence des simulations d'intérêt composé. Les champs **params** et **result** sont des JSON sérialisés.

Champ	Type SQL	Description
id	CHAR(25) PK	CUID
userId	CHAR(25) FK	Propriétaire
asset	VARCHAR	Actif simulé
params	TEXT (JSON)	{ initialAmount, monthlyInvestment, months, annualReturn, volatility, mode }
result	TEXT (JSON)	{ totalInvested, finalBalance, totalGains, roi }
monthlyData	TEXT (JSON)	Array mensuel [{ month, balance, invested, monthlyContribution, gainLoss }]

PORTFOLIO_SNAPSHOT — Instantané de portefeuille

Snapshot mensuel du capital de l'utilisateur.

Champ	Type SQL	Contraintes
id	CHAR(25)	PK
userId	CHAR(25)	FK → User
capital	FLOAT8	NOT NULL
month	SMALLINT	NOT NULL (1–12)
year	SMALLINT	NOT NULL

Contrainte unique : [userId, month, year]

COURSE — Cours de formation

Module du LMS intégré à Alvio. Chaque cours contient plusieurs leçons.

Champ	Type SQL	Description			
id	CHAR(25) PK	CUID			
title	TEXT	Titre du cours			
description	TEXT	Description			
level	ENUM	DEBUTANT \	INTERMEDIAIRE \	AVANCE \	EXPERT
category	VARCHAR	Ex : analyse_tec hnique, gestion_ du_risque			
thumbnail	TEXT NULL	URL image de couverture			
duration	INT	Durée totale en minutes			
totalLessons	INT	Nombre de leçons			
order	INT	Ordre d'affichage dans le catalogue			
isPublished	BOOL DEFAULT false	Visibilité			

Index : [level], [isPublished]

LESSON — Leçon

Contenu unitaire d'un cours. Type VIDEO : lien YouTube ; ARTICLE : contenu Markdown ; QUIZ : contenu JSON avec questions/réponses.

Champ	Type SQL	Description		
id	CHAR(25) PK	CUID		
courseId	CHAR(25) FK → Course	Cours parent (CASCADE DELETE)		
title	TEXT	Titre de la leçon		
description	TEXT	Résumé		
videoUrl	TEXT NULL	URL vidéo (YouTube embed)		
content	TEXT	Corps de la leçon		
duration	INT	Durée estimée en minutes		
order	INT	Ordre dans le cours		
type	ENUM	VIDEO \	ARTICLE \	QUIZ

Le MCD ci-dessus représente les 11 entités et leurs relations sémantiques. Les principales associations :

- **USER** → **SIGNAL** (1,N) : un utilisateur possède plusieurs signaux
- **USER** → **STRATEGY** (1,N) : un utilisateur crée plusieurs stratégies
- **USER** → **REPORT** (1,N) : un utilisateur a un rapport par couple (mois, année)
- **USER** → **WEBAUTHN** (1,N) : un utilisateur peut enregistrer plusieurs clés FIDO2
- **COURSE** → **LESSON** (1,N) : un cours contient plusieurs leçons
- **USER + LESSON** → **USER_PROGRESS** (N,N résolu) : table de jointure avec attributs

Choix de conception notables :

- **Signal.direction** inclut **BUY**, **SELL**, **HOLD** dans le schéma pour une compatibilité ascendante, mais la logique applicative est **long-only** : seuls des BUY sont créés ; la sortie passe par **status=CLOSED** sur le BUY.

- **AuthLog.userId** est nullable pour capturer les tentatives sur des comptes inexistants sans violer la contrainte de clé étrangère.

Modèle Logique de Données (MLD)

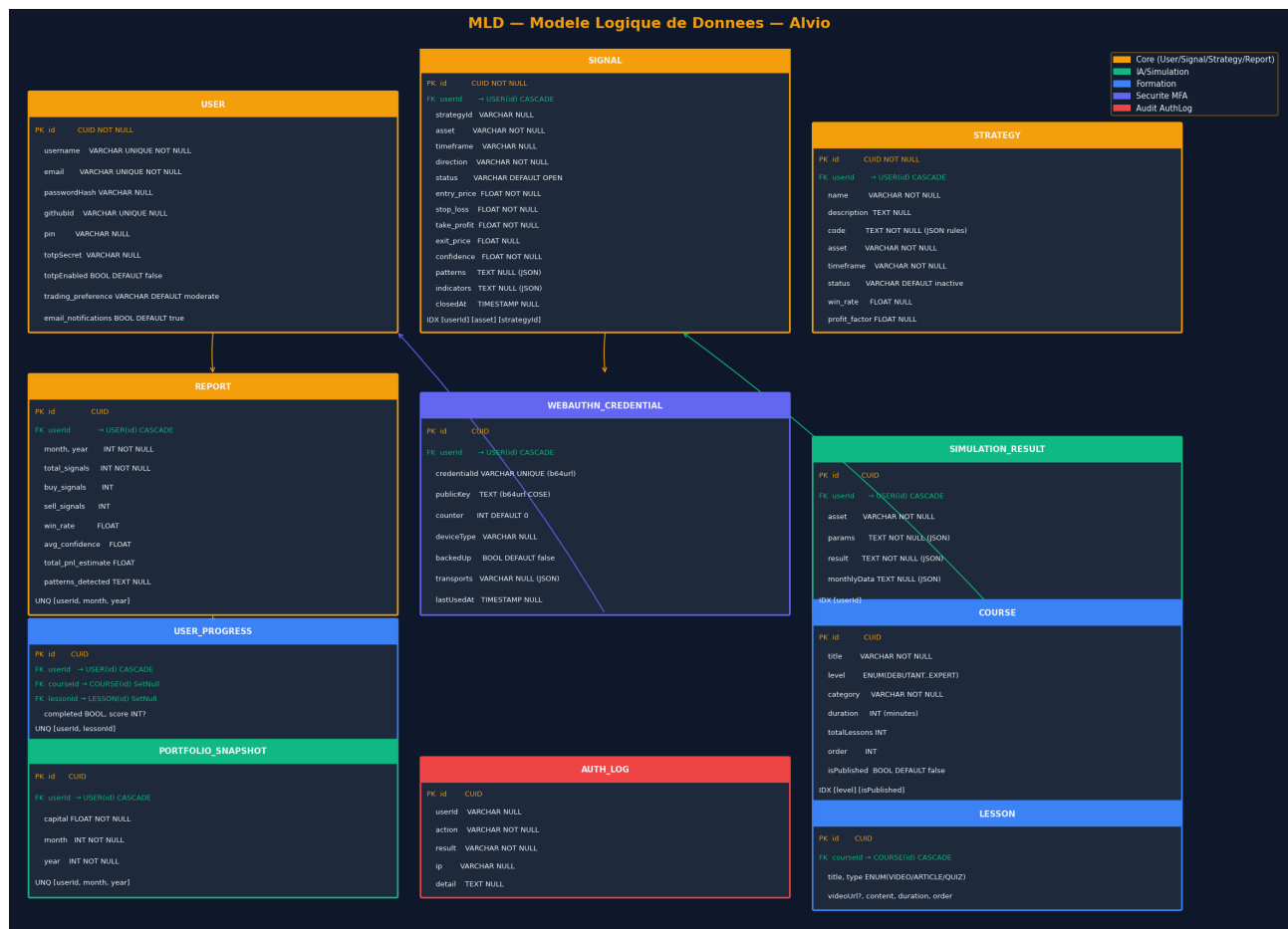


Figure — MLD Alvio (mld.png)

Le MLD traduit le MCD en tables relationnelles avec les clés primaires (PK), les clés étrangères (FK) et les contraintes UNIQUE. Points notables :

- Toutes les PK sont des **CUID** (Collision-resistant Unique IDentifier) — plus sûrs que les entiers auto-incrémentés (non prédictibles, non séquentiels).
- Les champs JSON (`patterns`, `indicators`, `params`, `result`, `monthlyData`, `patterns_detected`, `indicators_used`) sont stockés en TEXT sérialisé — PostgreSQL `JSONB` est possible mais non utilisé ici pour maintenir la compatibilité SQLite en développement.
- Les FK sur `UserProgress` → `Course` et `UserProgress` → `Lesson` utilisent `SetNull` (et non `CASCADE`) pour préserver l'historique de progression si un cours est supprimé.

Modèle Physique de Données (MPD)



Figure — MPD Alvio (mpd.png)

Le MPD détaille les types SQL PostgreSQL 15, les valeurs par défaut et les index créés automatiquement par Prisma. Index significatifs :

Table	Index	Justification
Signal	[userId]	Requêtes fréquentes par utilisateur
Signal	[asset]	Filtrage par actif

Table	Index	Justification
Signal	[strategyId, asset, direction, status]	Déduplication BUY OPEN
Course	[level], [isPublished]	Filtrage catalogue formation
Lesson	[courseId]	Jointure Course → Lesson

7. Modélisation UML

Diagramme de cas d'utilisation

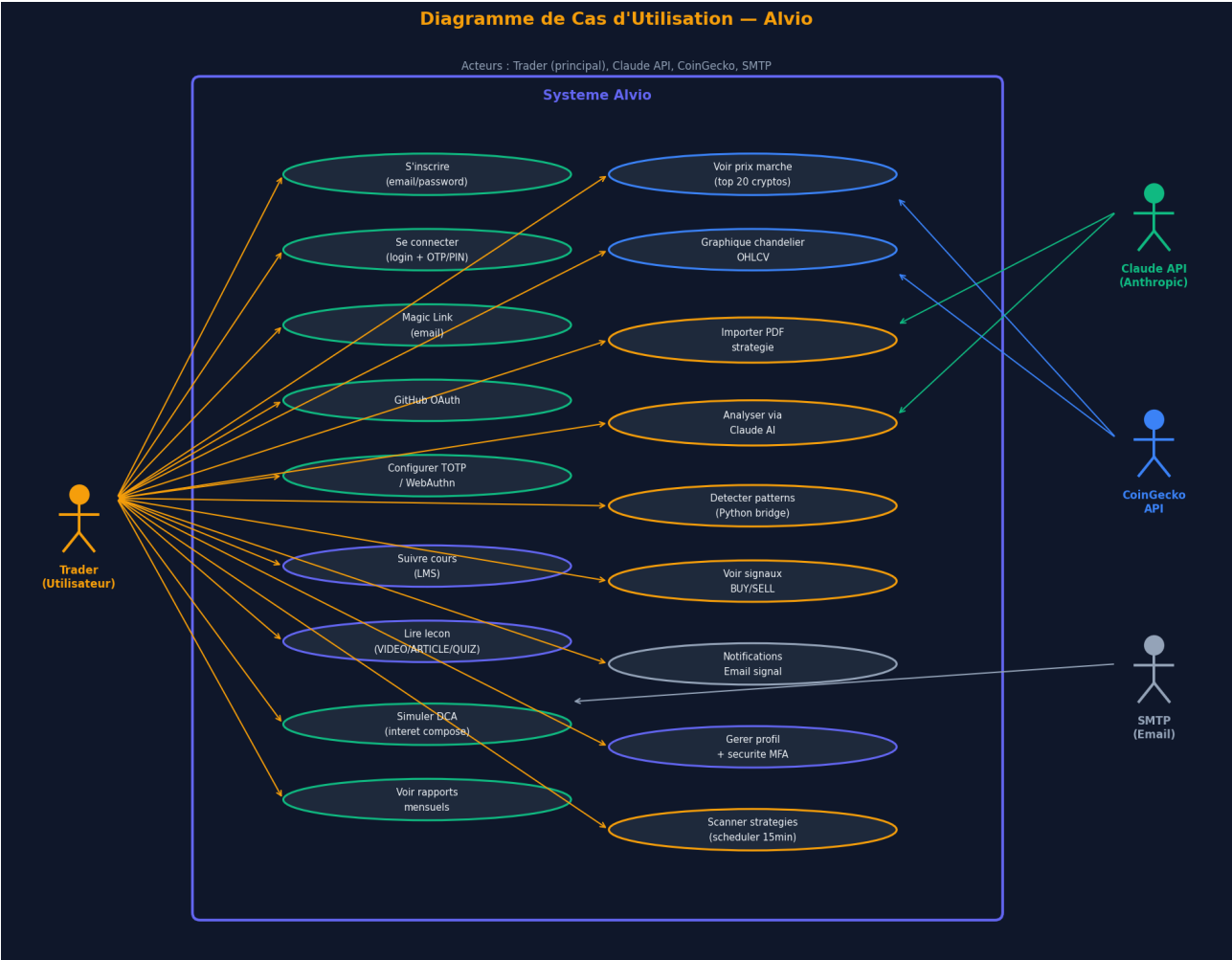


Figure — Cas d'utilisation Alvio (use_cases.png)

Acteurs

Acteur	Rôle dans le système
Trader (acteur principal)	Utilisateur authentifié — déclenche toutes les actions métier
Claude API (Anthropic)	Acteur externe — reçoit le texte extrait du PDF, retourne les StrategyRules JSON
CoinGecko API	Acteur externe — fournit les données de marché (top 20, OHLCV)
SMTP (Gmail)	Acteur externe — achemine les OTP, magic links et notifications de signal

Cas d'utilisation principaux

Authentification (6 use cases)

- S'inscrire avec email/password → `POST /auth/register`
- Se connecter (login + OTP email ou TOTP ou PIN) → `POST /auth/login` + `POST /auth/2fa/verify`
- Se connecter via Magic Link → `POST /auth/magic-link/request` + `GET /auth/magic-link/verify`
- Se connecter via GitHub OAuth → `GET /auth/github` → callback → redirect frontend
- Configurer TOTP (QR code, confirmation) → `POST /mfa/totp/enroll` + `POST /mfa/totp/verify`
- Enregistrer une clé WebAuthn → `POST /mfa/webauthn/register-options` + `register-verify`

Marchés (2 use cases)

- Voir les prix du marché (top 20 cryptos, sparklines) → `GET /markets` — cache Redis 60 s
- Consulter le graphique chandelier OHLCV d'un actif → `GET /markets/:id/ohlcv`

IA & Signaux (4 use cases)

- Importer un PDF de stratégie → `POST /strategies/import` (Multer, 10 MB max, pdf-parse v2)
- Analyser une stratégie via Claude AI → `POST /strategies/:id/analyze` → claude-sonnet-4-6
- Déclencher la détection de patterns Python → `POST /signals/generate` → spawn stdin/stdout
- Consulter les signaux BUY → `GET /signals` (50 derniers) + `GET /signals/statistics`

Formation (2 use cases)

- Parcourir le catalogue de cours et lire une leçon → `GET /formation/courses`
- Marquer une leçon comme complète → `POST /formation/progress`

Outils (3 use cases)

- Simuler une stratégie DCA → `POST /simulator/dca`
- Consulter les rapports mensuels → `GET /reports/:year/:month`
- Gérer son profil et sa sécurité MFA → `/profile` + `/profile/security`

Diagramme de classes



Figure — Diagramme de classes NestJS (classes.png)

Le diagramme de classes représente les **4 couches de l'architecture NestJS** :

Couche 1 — Controllers (HTTP)

Reçoivent les requêtes HTTP, valident les DTOs via **class-validator**, extraient l'identité de l'utilisateur depuis le token JWT (**@GetUser()**), et délèguent immédiatement au service correspondant.

```

AuthController      → /auth/*
StrategiesController → /strategies
SignalsController   → /signals
MarketsController    → /markets
FormationController  → /formation
SimulatorController → /simulator
ReportsController    → /reports/:year/:month
AIController         → /ai/*
TOTPController       → /mfa/totp/*
WebAuthnController   → /mfa/webauthn/*
MetricsController    → /metrics (restreint par IP)
AppController        → /health

```

Chaque controller est décoré **@UseGuards(JwtAuthGuard)** sur les routes protégées — seul **AuthController** expose des endpoints publics.

Couche 2 — Services métier

Contiennent la logique applicative. Dépendances injectées via le constructeur (**PrismaService**, **RedisService**, **AIService**, etc.).

AuthService (519 lignes) — orchestrateur du système MFA :

TypeScript

```
// src/auth/auth.service.ts
async login(dto: LoginDto, res: Response, ip?: string): Promise<void> {
  // 1. Trouver l'utilisateur → Prisma
  // 2. Vérifier le verrou Redis (locked:{userId})
  // 3. bcrypt.compare(password, passwordHash)
  // 4. recordFailure() si échec → incrémente fail:login:{userId}
  // 5. Si succès : générer OTP → Redis preauth:{token} (TTL 600s)
  //    → EmailService.sendOTP() → retourner { preAuthToken }
}
```

StrategiesService — pipeline PDF → Claude :

TypeScript

```
// src/strategies/strategies.service.ts
async importFromDocument(file: Express.Multer.File, dto, userId: string) {
  const rawText = await this.extractText(file); // pdf-parse v2
  const truncated = rawText.slice(0, 15_000); // max 15 000 chars
  const rules = await this.aiService.analyzeStrategyDocument(truncated);
  return this.prisma.strategy.create({ data: { ...dto, userId, code: JSON.stringify(rules) } });
}
```

MarketsService — cache anti-429 :

TypeScript

```
// src/markets/markets.service.ts — double clé Redis
private async cacheSet<T>(key: string, value: T, ttl: number): Promise<void> {
  const s = JSON.stringify(value);
  await this.redis.set(key, s, ttl); // clé fraîche
  await this.redis.set(`${key}:stale`, s, 3_600); // fallback si 429
}
```

Couche 3 — Services infrastructure

Services techniques transverses, tous **@Injectable()** et **@Global()** pour les plus partagés (**PrismaService**, **RedisService**).

AIService — appel Claude :

TypeScript

```
// src/ai/ai.service.ts
const CLAUDE_MODEL = 'claude-sonnet-4-6'; // ligne 14 — hardcodé

async analyzeStrategyDocument(text: string): Promise<StrategyRules> {
  const client = new Anthropic({ apiKey: this.config.get('ANTHROPIC_API_KEY') });
  const response = await client.messages.create({
    model: CLAUDE_MODEL, max_tokens: 4096,
    system: SYSTEM_PROMPT,
    messages: [{ role: 'user', content: text }],
  });
  const raw = (response.content[0] as TextBlock).text;
  return JSON.parse(this.cleanJsonResponse(raw)); // retire ```json...```
}
```

PatternDetectionService — bridge Python via **spawn** :

TypeScript

```
// src/patterns/pattern-detection.service.ts
```

```
const proc = spawn('python3', ['/app/ai-module/pattern_detector.py'], {
  stdio: ['pipe', 'pipe', 'pipe'], // stdin / stdout / stderr
});
proc.stdin.write(JSON.stringify(input), 'utf-8');
proc.stdin.end();
// → lecture stdout → JSON.parse() → PatternDetectionResult
// → { global_status: 'ENTRY_SIGNAL' | 'EXIT_SIGNAL' | 'NO_SIGNAL', ... }
```

TOTPSERVICE — chiffrement AES-256-GCM :

TypeScript

```
// src/mfa/totp/totp.service.ts
// Stockage du secret chiffré pour limiter l'exposition en cas de fuite BDD
private encrypt(text: string): string {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', this.key, iv);
  // → { iv, ciphertext, authTag } sérialisé en base64
}
```

Couche 4 — Données & Externes

- **PrismaService** : `@Global()`, wrapping `PrismaClient`, expose tous les modèles (`this.prisma.signal.findMany(...)`) avec requêtes paramétrées (protection SQLi native).

- **RedisService** : `@Global()`, wrapping `ioredis`, méthodes `get/set/del/incr`.

- **EmailService** : `nodemailer` + SMTP Gmail, fire-and-forget (non bloquant).

- **APIs externes** : CoinGecko REST v3 (via `MarketsService`) et Anthropic Claude (via `AIService`).

Diagramme de séquence — Création de stratégie IA

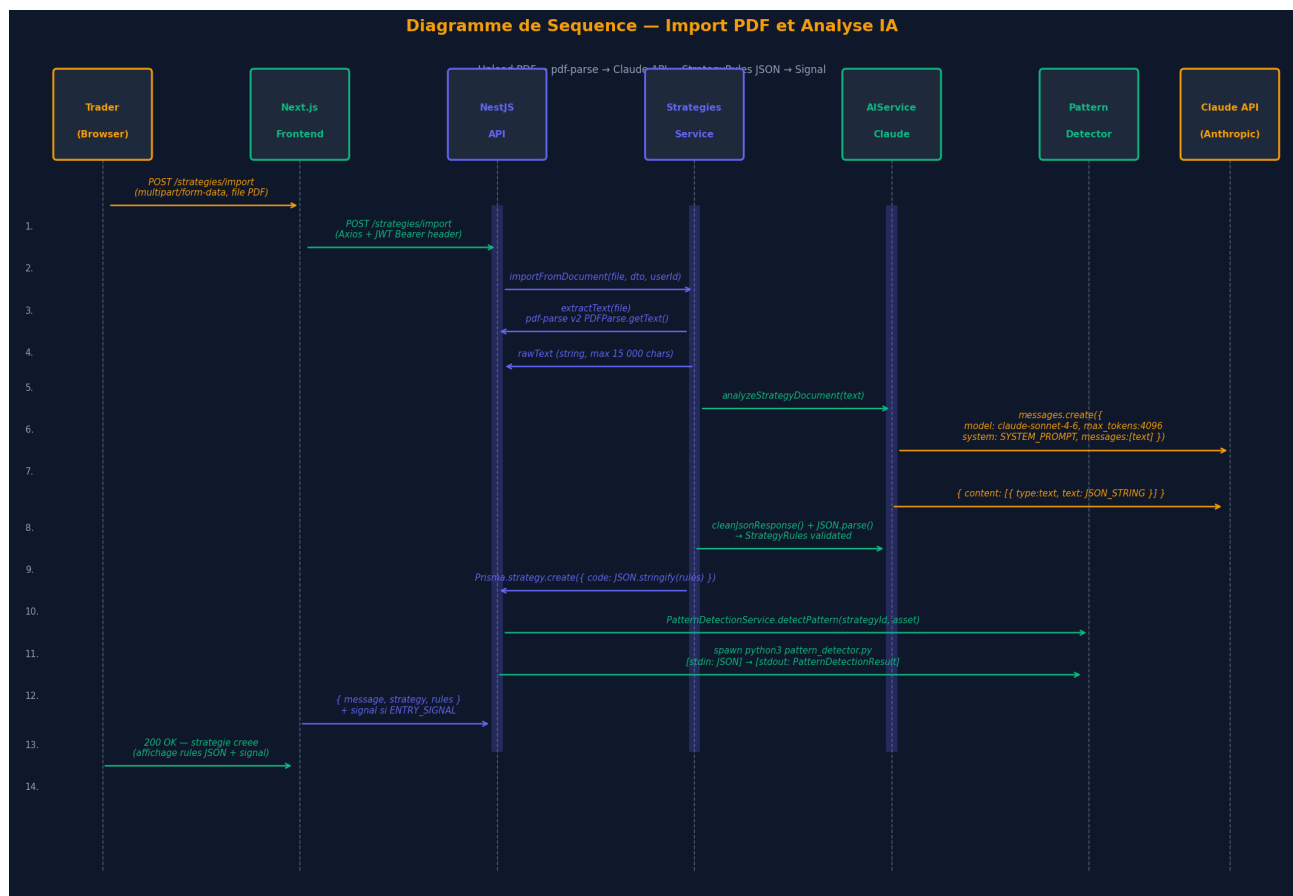


Figure — Séquence création stratégie (seq_strategy.png)

Ce diagramme illustre le flux complet d'import d'un PDF de stratégie et sa transformation en signal de trading.

Étapes clés :

- 1. Upload** — Le Trader soumet un fichier PDF via `POST /strategies/import` (multipart/form-data, limite 10 MB, Multer `memoryStorage`).
- 2. Extraction texte** — `StrategiesService.extractText()` appelle `pdf-parse v2` avec l'API classe : `typescript // API v2 (classe, pas fonction) – breaking change vs v1 const parser = new PDFParse({ data: file.buffer }); const rawText = await parser.getText(); // Tronqué à 15 000 chars avant envoi à Claude`
- 3. Analyse Claude** — `AIService.analyzeStrategyDocument()` envoie le texte à `claude-sonnet-4-6` avec un `SYSTEM_PROMPT` imposant un JSON strict. La réponse est nettoyée (`cleanJsonResponse`) puis validée (`validateStrategyRules`).
- 4. Persistance** — La stratégie est créée en base : `Prisma.strategy.create({ code: JSON.stringify(rules) })`.
- 5. Détection immédiate** — `PatternDetectionService.detectPattern()` est appelé immédiatement : récupération des OHLCV via `MarketsService`, puis `spawn python3 pattern_detector.py (stdin/stdout)`.
- 6. Signal** — Si `global_status === 'ENTRY_SIGNAL'`, un signal BUY est

créé en base avec `SignalsService.createSignal()`.

Diagramme de séquence — Authentification multi-facteurs

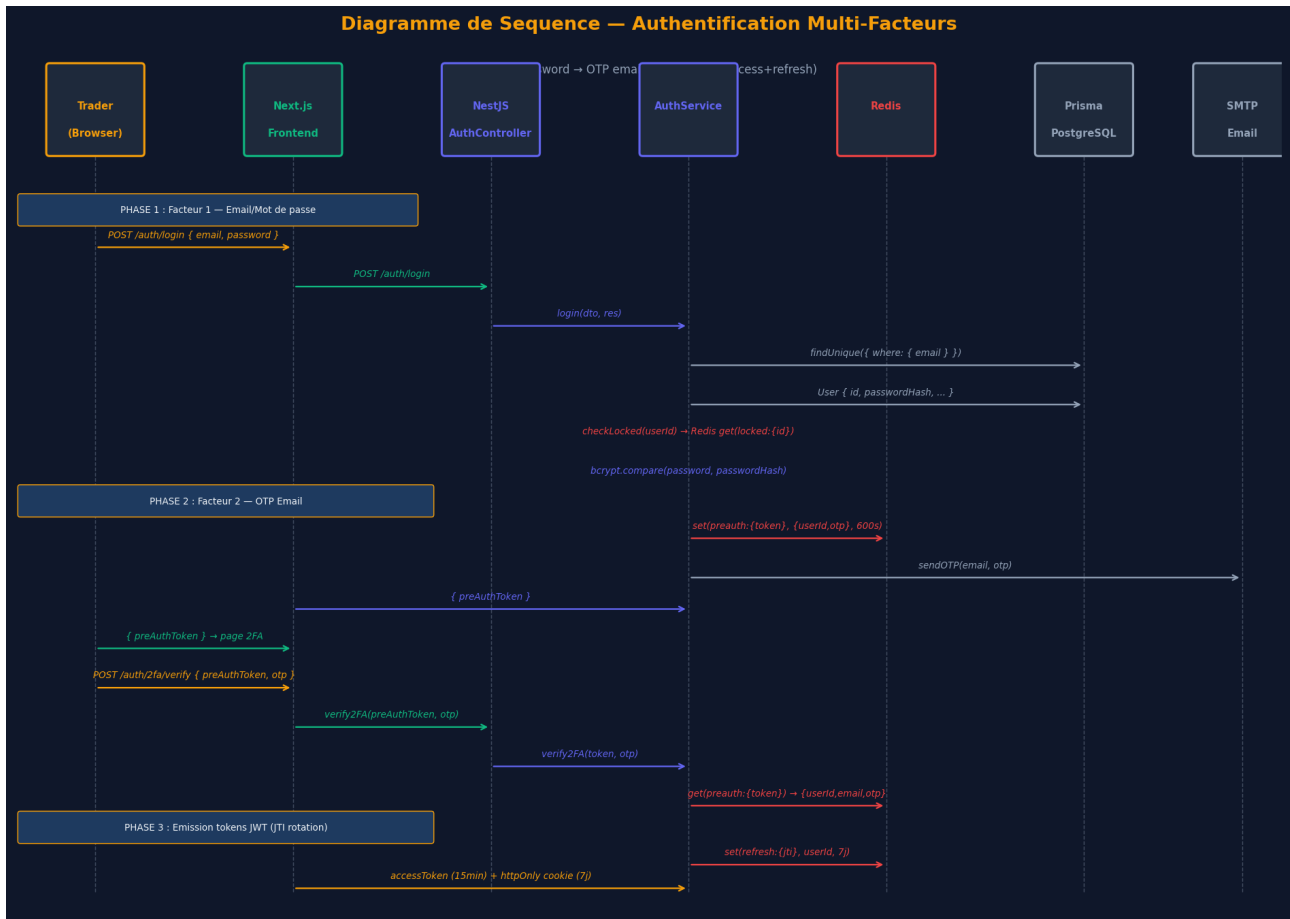


Figure — Séquence authentification (seq_auth.png)

Ce diagramme couvre le flux login complet en 3 phases successives.

Phase 1 — Facteur 1 : Email / Mot de passe

1. `POST /auth/login { email, password }` → `AuthService.login()`
2. `Prisma.user.findUnique({ where: { email } })` → récupération du compte
3. Vérification du verrou : `Redis.get('locked:{userId}')` — si présent, `HTTP 423`
4. `bcrypt.compare(password, passwordHash)` — si KO :
`recordFailure()` → incrémente `fail:login:{userId}` → verrou si ≥ 3 échecs
5. Si OK : génération d'un OTP 6 chiffres → `Redis.set('preauth:{token}', {userId, otp}, 600)`
→ `EmailService.sendOTP(email, otp)` → retour `{ preAuthToken }`

Phase 2 — Facteur 2 : OTP Email

6. `POST /auth/2fa/verify { preAuthToken, otp }` → `AuthService.verify2FA()`
7. `Redis.get('preauth:{token}')` → récupération de `{userId, email, otp_attendu}`

8. Comparaison OTP — si KO : `recordFailure()` → verrou si ≥ 3 échecs

9. Si OK : le flow continue (PIN ou émission directe des tokens selon la config)

Phase 3 — Émission des tokens JWT (JTI rotation)

10. `jti = crypto.randomUUID()` — identifiant unique du refresh token

11. `accessToken` : JWT signé, `expiresIn: '15m'`, transporté en mémoire côté client

12. `refreshToken` : JWT signé, `expiresIn: '7d'`, stocké :

- Redis : `set('refresh:{jti}', userId, 7 24 3600)`

- Cookie `httpOnly` : `SameSite: 'strict', Secure: true`

13. À chaque `POST /auth/refresh` : le `jti` est révoqué dans Redis avant

d'en émettre un nouveau — rotation garantissant qu'un refresh token volé est invalidé dès sa détection.

Sécurité du flux

Menace	Contre-mesure
Brute-force login	Verrou Redis après 3 échecs — <code>locked:{userId}</code> TTL 1800 s
Brute-force IP	Compteur <code>fail:ip:{ip}</code> — alerté via <code>LoggingService</code>
Vol du refresh token	JTI rotation — token révocable individuellement
CSRF	Cookie <code>SameSite: strict</code> + absence du token dans les URL
Session fixation	Nouveau <code>jti</code> à chaque refresh — pas de réutilisation

8. Architecture Logicielle

Vue d'ensemble

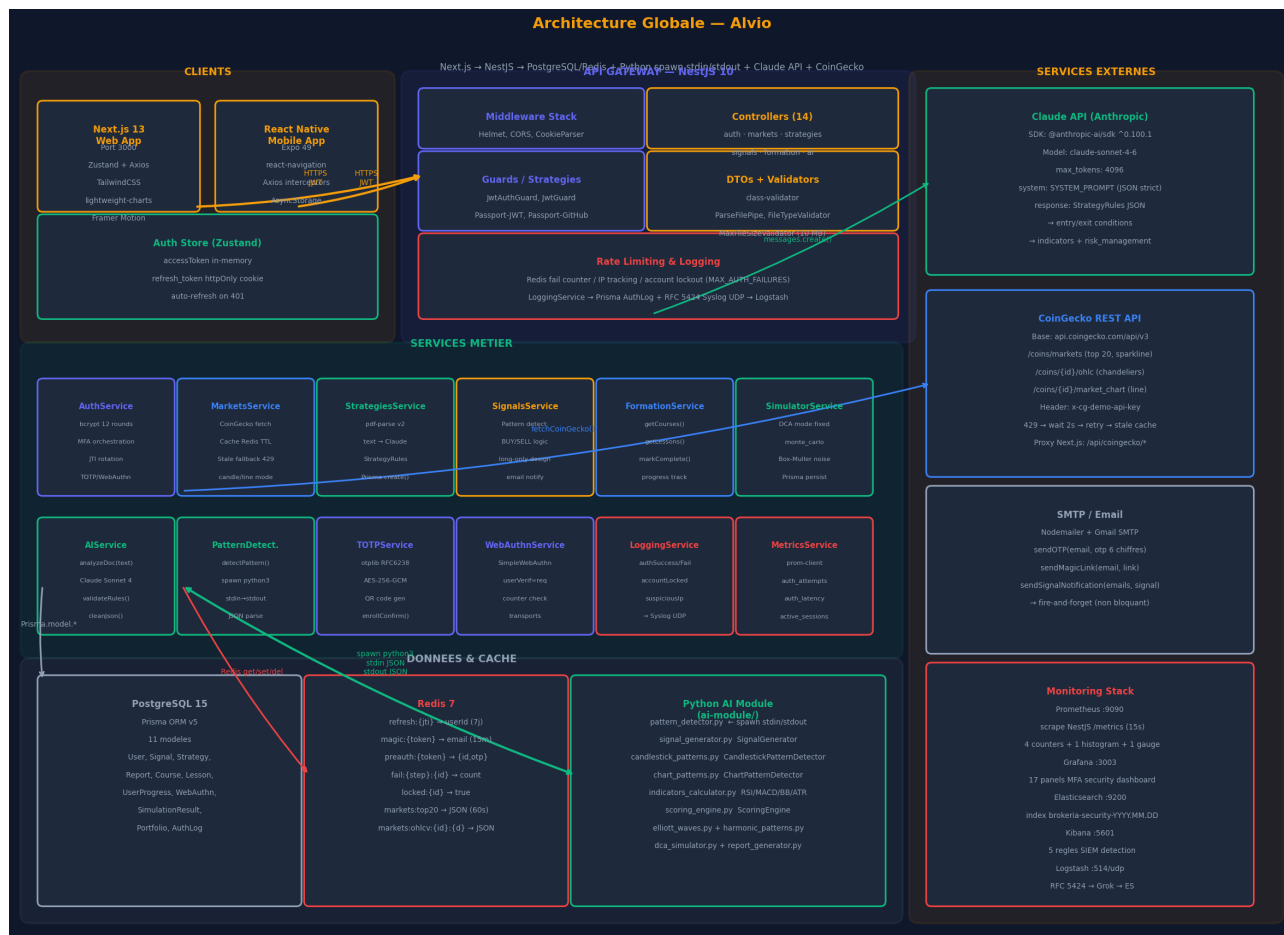
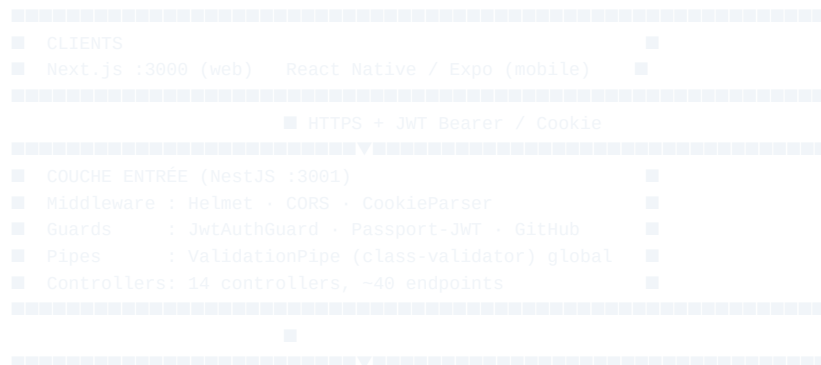
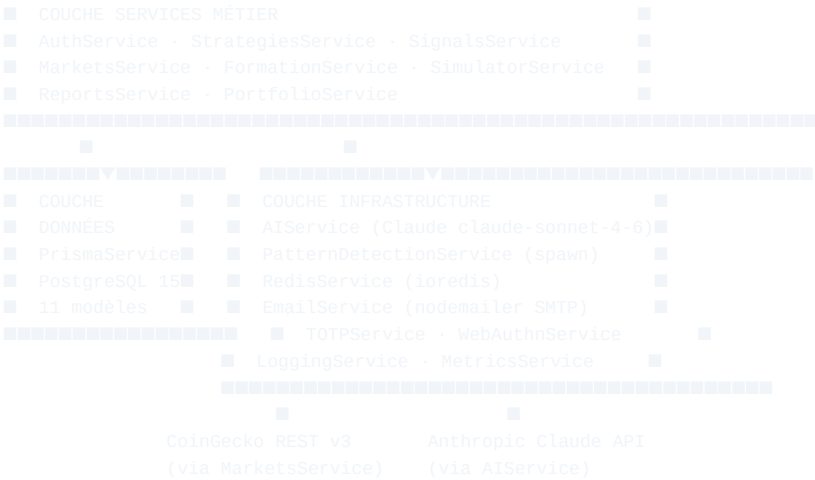


Figure — Architecture globale Alvio (architecture.png)

Alvio repose sur une architecture **en couches strictement séparées**, conforme aux principes NestJS (Inversion of Control, injection de dépendances, modules isolés). Chaque couche ne connaît que la couche immédiatement inférieure.





Principes d'architecture appliqués

Principe	Implémentation concrète
Séparation des couches	Contrôleurs → ne font qu'appeler un service, jamais Prisma directement
Injection de dépendances	Tout est <code>@Injectable()</code> — constructeur typé, aucun <code>new Service()</code> en dehors du framework
Services globaux	<code>PrismaService</code> et <code>RedisService</code> décorés <code>@Global()</code> — disponibles sans import dans chaque module
Modules isolés	Chaque domaine métier est un <code>@Module()</code> indépendant (<code>AuthModule</code> , <code>MarketsModule</code> , etc.)
Validation aux frontières	<code>ValidationPipe({ whitelist: true, transform: true })</code> appliqué globalement — aucun champ non déclaré ne traverse les DTOs

Vue des endpoints (routes réelles)

`/auth` — Authentification

Méthode	Route	Guard	Description
POST	<code>/auth/register</code>	—	Inscription email/password
POST	<code>/auth/login</code>	—	Login → OTP email (preAuthToken)
POST	<code>/auth/magic-link/request</code>	—	Envoi magic link par email
POST	<code>/auth/magic-link/verify</code>	—	Vérification du token magic link
GET	<code>/auth/github</code>	Passport('github')	Redirection OAuth GitHub
GET	<code>/auth/github/callback</code>	Passport('github')	Callback → redirect frontend
POST	<code>/auth/2fa/verify</code>	—	Vérification OTP email (2ème facteur)

Méthode	Route	Guard	Description
POST	<code>/auth/pin/verify</code>	—	Vérification PIN (3ème facteur)
POST	<code>/auth/pin/setup</code>	—	Configuration initiale du PIN
POST	<code>/auth/set-password</code>	—	Définition mot de passe (post-GitHub/MagicLink)
POST	<code>/auth/refresh</code>	Cookie	Rotation du refresh token (JTI)
POST	<code>/auth/logout</code>	—	Révocation du refresh token dans Redis
POST	<code>/auth/dev-login</code>	—	Login dev-only (désactivé en prod)
GET	<code>/auth/profile</code>	JwtAuthGuard	Profil de l'utilisateur connecté

`/strategies` — Stratégies de trading

Méthode	Route	Description
GET	<code>/strategies</code>	Liste des stratégies de l'utilisateur
POST	<code>/strategies/:id/analyze</code>	Analyser une stratégie existante via Claude
POST	<code>/strategies/import</code>	Upload PDF → pdf-parse v2 → Claude → StrategyRules

`/signals` — Signaux de trading

Méthode	Route	Description
GET	<code>/signals</code>	50 derniers signaux de l'utilisateur
GET	<code>/signals/recent</code>	5 signaux les plus récents
POST	<code>/signals</code>	Création manuelle d'un signal
GET	<code>/signals/statistics</code>	Statistiques globales
POST	<code>/signals/generate</code>	Génération via PatternDetectionService → spawn Python
POST	<code>/signals/scan-now</code>	Déclenchement immédiat du scheduler

`/markets` — Données de marché

Méthode	Route	Cache Redis	Description
GET	<code>/markets</code>	60 s	Top 20 cryptos (CoinGecko <code>/coins/markets</code>)
GET	<code>/markets/:id</code>	30 s	Détail d'une crypto
GET	<code>/markets/:id/ohlcv</code>	Variable (60–1800 s)	Données OHLCV chandeliers ou line

`/formation` — LMS

Méthode	Route	Description
GET	<code>/formation/courses</code>	Catalogue des cours publiés + progression
GET	<code>/formation/courses/:id</code>	Détail d'un cours + ses leçons
GET	<code>/formation/lessons/:id</code>	Contenu d'une leçon
POST	<code>/formation/progress</code>	Marquer une leçon comme complète
GET	<code>/formation/my-progress</code>	Progression globale de l'utilisateur

Autres endpoints

Controller	Route	Description
SimulatorController	POST <code>/simulator/dca</code>	Simulation DCA (fixed / monte_carlo)
SimulatorController	GET <code>/simulator/history</code>	Historique des simulations
ReportsController	GET <code>/reports/:year/:month</code>	Rapport mensuel calculé
PortfolioController	GET <code>/portfolio/history</code>	Historique snapshots
PortfolioController	GET <code>/portfolio/stats</code>	Statistiques portefeuille
AIController	POST <code>/ai/analyze</code>	Analyse directe via Claude
AIController	POST <code>/ai/detect-patterns</code>	Détection patterns (debug)
AIController	POST <code>/ai/generate-signal</code>	Génération signal (debug)
TOTPController	POST <code>/mfa/totp/enroll</code>	Init enrôlement TOTP (QR code)
TOTPController	POST <code>/mfa/totp/verify</code>	Confirmation enrôlement
TOTPController	DELETE <code>/mfa/totp</code>	Désactivation TOTP
WebAuthnController	POST <code>/mfa/webauthn/register-options</code>	Options d'enregistrement FIDO2
WebAuthnController	POST <code>/mfa/webauthn/register-verify</code>	Vérification enregistrement
WebAuthnController	POST <code>/mfa/webauthn/auth-options</code>	Options d'authentification
WebAuthnController	POST <code>/mfa/webauthn/auth-verify</code>	Vérification authentification
MetricsController	GET <code>/metrics</code>	Métriques Prometheus (IP restreinte)
AppController	GET <code>/health</code>	Healthcheck → 200 OK

Sécurité & sécurisation des endpoints

Garde d'authentification — `JwtAuthGuard`

Toutes les routes métier (signaux, stratégies, marchés, formation, rapports, simulateur, profil) sont protégées par `@UseGuards(JwtAuthGuard)`.

TypeScript

```
// src/auth/guards/jwt-auth.guard.ts
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  // Passport-JWT extrait le token du header Authorization: Bearer <token>
  // Vérifie la signature + expiration (15 min)
  // Injecte { sub: userId, email } dans req.user
}
```

Seules les routes d'authentification (`/auth/register`, `/auth/login`, `/auth/magic-link/*`, `/auth/2fa/verify`, `/auth/pin/verify`, `/auth/github`) sont publiques.

Authentification multi-facteurs (MFA)

Le système MFA est entièrement géré dans `AuthService` sans bibliothèque d'identité tierce. Trois facteurs successifs selon la configuration :

```
Facteur 1 : email / password → bcrypt.compare (12 rounds)
Facteur 2 : OTP 6 chiffres → Redis preauth:{token} (TTL 600 s)
    OU TOTP RFC 6238 → otplib.totp.verify()
    OU Magic Link → Redis magic:{token} (TTL 900 s)
    OU GitHub OAuth → passport-github2
Facteur 3 : PIN bcrypt → Redis pinauth:{token} (TTL 300 s)
    OU WebAuthn FIDO2 → @simplewebauthn/server (counter anti-replay)
```

Verrouillage de compte

TypeScript

```
// src/auth/auth.service.ts — seuils configurables via .env
// MAX_AUTH_FAILURES=3, LOCK_TTL_SECONDS=1800

private async recordFailure(userId: string, step: string, ip?: string): Promise<void> {
  const key = `fail:${step}:${userId}`;
  const count = parseInt(await this.redis.get(key) ?? '0') + 1;
  await this.redis.set(key, count.toString(), this.lockTtl); // reset TTL à chaque échec
  if (ip) await this.redis.incr(`fail:ip:${ip}`); // compteur IP
  if (count >= this.maxFailures) {
    await this.redis.set(`locked:${userId}`, 'true', this.lockTtl);
    this.logging.accountLocked(userId, ip);
    throw new HttpException('Compte bloqué temporairement', 423);
  }
}
```

Tokens JWT — rotation JTI

TypeScript

```
// src/auth/auth.service.ts
private async issueTokens(userId: string, email: string, res: Response): Promise<string> {
  const jti = crypto.randomUUID(); // identifiant unique du refresh token

  const accessToken = this.jwtService.sign(
    { sub: userId, email },
    { expiresIn: '15m' } // access token court
  );
  const refreshToken = this.jwtService.sign(
    { sub: userId, jti },
    { expiresIn: '7d' }
  );

  await this.redis.set(`refresh:${jti}`, userId, 7 * 24 * 3600); // révocable
```

```

    res.cookie('refresh_token', refreshToken, {
      httpOnly: true, // non accessible en JavaScript
      secure: true, // HTTPS uniquement
      sameSite: 'strict',
      maxAge: 7 * 24 * 3600 * 1000,
    });

    return accessToken; // renvoyé dans le corps de la réponse
  }
}

```

À chaque **POST /auth/refresh**, l'ancien **jwt** est supprimé de Redis avant l'émission d'un nouveau refresh token — garantissant la révocabilité unitaire.

Gestion des vulnérabilités

Injection SQL (SQLi)

Traitement : Prisma génère exclusivement des requêtes **paramétrées** — aucune concaténation de chaîne dans les requêtes. Même si une entrée utilisateur contient du SQL, elle est transmise comme valeur de paramètre et jamais interpolée dans la requête.

TypeScript

```

// Exemple — src/signals/signals.service.ts
// L'identifiant userId vient du JWT (pas de l'utilisateur directement)
const signals = await this.prisma.signal.findMany({
  where: { userId }, // Prisma génère : SELECT ... WHERE "userId" = $1
  orderBy: { createdAt: 'desc' },
  take: 50,
});
// Impossible d'injecter via userId car il est toujours extrait du token JWT signé

```

Cross-Site Scripting (XSS)

Traitement côté frontend (Next.js / React) : React échappe automatiquement toute valeur interpolée dans le JSX (**{value}** → entités HTML). Les données de l'API (noms de stratégies, symboles d'actifs, résumés de rapport) sont toujours rendues via des nœuds React, jamais via **dangerouslySetInnerHTML**.

Côté backend : le header **X-Content-Type-Options: nosniff** est positionné par **Helmet** (middleware global NestJS) — empêche le MIME-sniffing utilisé dans certaines variantes XSS.

TypeScript

```

// src/main.ts
app.use(helmet()); // X-Content-Type-Options, X-Frame-Options, CSP de base
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));

```

Cross-Site Request Forgery (CSRF)

Modèle d'auth : l'access token JWT est stocké **en mémoire côté client** (jamais dans **localStorage** ni dans un cookie accessible en JS). Le cookie **refresh_token** est **httpOnly** + **SameSite: strict** — il n'est pas transmis sur des requêtes cross-origin.

Les requêtes API authentifiées envoient le JWT dans le header **Authorization: Bearer <token>** — un attaquant ne peut pas forger ce header depuis un site tiers (la Same-Origin Policy l'en empêche).

Résumé : l'architecture sans token CSRF explicite est sécurisée car :

- 1. Le refresh cookie a `SameSite: strict` — non transmis cross-site
- 2. Le access token est en mémoire — inaccessible aux scripts tiers
- 3. Les mutations d'état (POST, PATCH, DELETE) requièrent le header JWT

Autres protections actives

Menace	Contre-mesure
Brute-force	Verrouillage Redis après 3 échecs (par userId + par IP)
Vol de refresh token	JTI rotation — révocable individuellement
Replay WebAuthn	Vérification du <code>counter</code> (croissance stricte)
Exposition clé TOTP	Secret chiffré AES-256-GCM en base — clé dérivée de <code>ENCRYPTION_KEY</code>
Enumération d'emails	Réponse identique si email existant ou non (<code>200 OK</code> + message générique)
Replay magic link	Suppression du token Redis à la première utilisation

Gestion des dépendances et configuration

Variables d'environnement (`.env` backend`)

Bash / Shell

```
# Base de données
DATABASE_URL="postgresql://brokeria_user:BrokerIA_SecurePass2026@localhost:5432/brokeria"

# JWT
JWT_SECRET=... # secret de signature access + refresh tokens

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# Auth
MAX_AUTH_FAILURES=3
LOCK_TTL_SECONDS=1800
FRONTEND_URL=http://localhost:3000

# APIs externes
ANTHROPIC_API_KEY=... # clé API Claude
COINGECKO_API_KEY=... # clé demo CoinGecko (header x-cg-demo-api-key)

# Email SMTP
SMTP_HOST=smtp.gmail.com
SMTP_USER=...
SMTP_PASS=... # App Password Gmail (pas le mot de passe du compte)

# Sécurité MFA
ENCRYPTION_KEY=... # 32 bytes hex — chiffrement AES-256-GCM des secrets TOTP
WEBAUTHN_RP_ID=localhost # Relying Party ID WebAuthn
WEBAUTHN_ORIGIN=http://localhost:3000
```

Dépendances backend critiques (versions réelles)

Package	Version	Rôle
@nestjs/common	^10.0.0	Framework
@prisma/client	^5.0.0	ORM typé PostgreSQL
@anthropic-ai/sdk	^0.100.1	Claude API
ioredis	^5.10.1	Client Redis
bcrypt	^5.1.0	Hash password/PIN (12 rounds)
pdf-parse	^2.4.5	Parsing PDF (API classe v2)
otplib	^12.0.1	TOTP RFC 6238
@simplewebauthn/server	^10.0.1	WebAuthn FIDO2
passport-github2	^0.1.12	GitHub OAuth
prom-client	^15.1.3	Métriques Prometheus
nodemailer	^8.0.2	Envoi emails SMTP
helmet	inclus NestJS	Headers sécurité HTTP
class-validator	^0.14.0	Validation DTOs
@nestjs/schedule	^6.1.3	Scheduler cron (scan signaux 15 min)

Dépendances frontend critiques

Package	Version	Rôle
next	13.4.0	Framework SSR (Pages Router)
zustand	^4.5.7	State management (auth store)
axios	^1.4.0	HTTP client + intercepteurs 401/refresh
lightweight-charts	^4.2.3	Graphiques OHLCV TradingView
framer-motion	^12.34.1	Animations
@simplewebauthn/browser	^10.0.0	WebAuthn côté navigateur

Workflow : parcours complet d'une requête

Exemple : GET /signals — récupération des 50 derniers signaux

Étape 1 — Client (Next.js)

TypeScript

```
// frontend-web/pages/signals/index.tsx (simplifié)
// Axios instance avec intercepteur 401 → auto-refresh
const { data } = await apiClient.get('/signals');
// Header automatiquement ajouté : Authorization: Bearer <accessToken>
```

Étape 2 — Middleware stack (NestJS)

La requête traverse dans l'ordre :

1. `helmet()` — positionne les headers sécurité
2. `cors()` — vérifie l'origine contre `FRONTEND_URL`
3. `cookieParser()` — parse les cookies (pour le refresh path)
4. `JwtAuthGuard` → `PassportStrategy('jwt')` :
 - extrait le token du header `Authorization: Bearer` - vérifie la signature HMAC-SHA256 avec `JWT_SECRET` - vérifie l'expiration (< 15 min) - injecte `{ sub: userId, email }` dans `req.user`

Étape 3 — Controller

TypeScript

```
// src/signals/signals.controller.ts
@Get()
@UseGuards(JwtAuthGuard)
async getSignals(@GetUser('sub') userId: string) {
  // userId extrait du token JWT — jamais fourni par le client directement
  return this.signalsService.getUserSignals(userId);
}
```

Étape 4 — Service

TypeScript

```
// src/signals/signals.service.ts
async getUserSignals(userId: string): Promise<Signal[]> {
  return this.prisma.signal.findMany({
    where: { userId },
    orderBy: { createdAt: 'desc' },
    take: 50,
  });
}
```

Étape 5 — Prisma → PostgreSQL

Prisma génère et exécute :

SQL

```
SELECT id, "userId", asset, direction, status, entry_price, stop_loss,
       take_profit, exit_price, confidence, patterns, indicators,
       "createdAt", "closedAt"
FROM "Signal"
WHERE "userId" = $1          -- paramètre, jamais interpolé
ORDER BY "createdAt" DESC
LIMIT 50;
```

Étape 6 — Réponse

Le tableau de `Signal[]` est sérialisé en JSON par NestJS et renvoyé avec `Content-Type: application/json` et le code HTTP approprié (200).

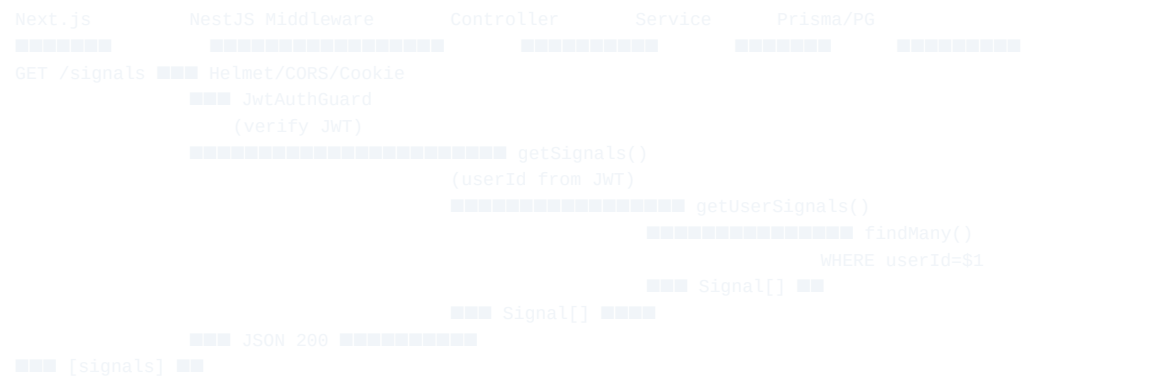
Le frontend Zustand met à jour le store :

TypeScript

```
// frontend-web/stores/signalStore.ts (simplifié)
const useSignalStore = create((set) => ({
  signals: [],
  fetchSignals: async () => {
    const { data } = await apiClient.get('/signals');
```

```
        set({ signals: data });
    },
  ));
```

Résumé du flux



9. Application Web

Vue d'ensemble

L'application web Alvio est construite avec **Next.js 13.4.0** en **Pages Router** (le dossier `frontend-web/pages/` contient l'ensemble des routes). Elle joue le rôle d'interface utilisateur complète : dashboard temps réel, gestion des stratégies IA, consultation des marchés avec graphiques chandeliers, module de formation (LMS), simulateur DCA, rapports mensuels et panneau de sécurité MFA.

Choix technologiques

Technologie	Version	Justification
Next.js 13 (Pages Router)	13.4.0	SSR natif pour le SEO des pages publiques, routing basé sur le système de fichiers
Zustand	^4.5.7	Store global léger — gestion du token JWT en mémoire (pas de <code>localStorage</code>)
Axios	^1.4.0	Client HTTP avec intercepteurs — refresh automatique du JWT sur erreur 401
Tailwind CSS	^3.3.0	Utility-first — cohérence avec la charte navy/amber/green/violet
lightweight-charts	^4.2.3	Bibliothèque TradingView — graphiques OHLCV chandelier hautes performances
framer-motion	^12.34.1	Animations des cartes, modales et transitions de page
@simplewebauthn/browser	^10.0.0	WebAuthn FIDO2 côté navigateur (enrôlement + authentification)
next-themes	^0.4.6	Basculement dark/light mode

Structure des pages (réelle)

```
frontend-web/pages/
  ├── index.tsx           ← Landing page (publique, indexée)
  ├── login.tsx / register.tsx ← Auth (publiques)
  ├── dashboard/index.tsx  ← Vue d'ensemble (protégée)
  ├── markets/index.tsx    ← Top 20 cryptos
  │   ├── markets/[id].tsx ← Détail + graphique OHLCV
  ├── strategies/index.tsx
  │   ├── strategies/new.tsx
  │   ├── strategies/import.tsx ← Upload PDF → Claude
  ├── signals/index.tsx
  │   ├── signals/[id].tsx
  ├── formation/index.tsx
  │   ├── formation/[courseId]/[lessonId].tsx
  ├── simulator/index.tsx ← DCA
  ├── reports/index.tsx
  ├── profile/index.tsx
  │   ├── profile/security.tsx ← MFA (TOTP, WebAuthn, PIN)
  ├── auth/               ← 2fa, magic, pin, setup-pin, locked,
```

```

■          github-callback, totp-setup/verify,
■          webauthn-setup, create-password
■■■ api/coingecko/          ← Proxy Next.js (3 routes)
    ■■■ markets.ts
    ■■■ coins/[id].ts
    ■■■ ohlc/[id].ts

```

Authentification — flux côté client

Gestion du token JWT (Zustand store)

L'access token est stocké **exclusivement en mémoire** dans le store Zustand (jamais dans `localStorage` ni `sessionStorage`). Le refresh token est dans un cookie `httpOnly` — invisible aux scripts JS.

TypeScript

```

// frontend-web/stores/authStore.ts (pattern Zustand)
interface AuthState {
  accessToken: string | null;
  user: { id: string; email: string; username: string } | null;
  setAuth: (token: string, user: AuthState['user']) => void;
  clearAuth: () => void;
}

const useAuthStore = create<AuthState>((set) => ({
  accessToken: null,
  user: null,
  setAuth: (token, user) => set({ accessToken: token, user }),
  clearAuth: () => set({ accessToken: null, user: null }),
}));

```

Intercepteurs Axios — refresh automatique

TypeScript

```

// frontend-web/lib/apiClient.ts
const apiClient = axios.create({ baseURL: '/api', withCredentials: true });

// Injecte le token JWT à chaque requête
apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Sur 401 : tente un refresh silencieux, puis relance la requête originale
let isRefreshing = false;
apiClient.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry) {
      original._retry = true;
      if (!isRefreshing) {
        isRefreshing = true;
        try {
          // POST /auth/refresh → cookie httpOnly envoyé automatiquement
          const { data } = await axios.post('/api/auth/refresh', {}, { withCredentials: true });
          useAuthStore.getState().setAuth(data.accessToken, data.user);
        } finally { isRefreshing = false; }
      }
      return apiClient(original); // relance la requête avec le nouveau token
    }
  }
);

```

```

    return Promise.reject(error);
  }
});

```

Protection des routes côté client

Les pages protégées utilisent un Higher-Order Component `withAuth` ou vérifient le store Zustand dans `getServerSideProps` :

TypeScript

```

// Pattern utilisé sur /dashboard, /signals, /strategies, etc.
export const getServerSideProps: GetServerSideProps = async (ctx) => {
  const token = ctx.req.cookies['refresh_token'];
  if (!token) return { redirect: { destination: '/login', permanent: false } };
  return { props: {} };
};

```

L'absence du cookie `refresh_token` redirige immédiatement vers `/login` côté serveur — la page protégée ne s'affiche jamais.

Flux login complet (UI → backend)

```

1. /login : saisie email + password → POST /auth/login
2. Backend répond : { preAuthToken } (access refusé jusqu'au 2ème facteur)
3. Redirection vers /auth/2fa (saisie OTP reçu par email)
4. POST /auth/2fa/verify { preAuthToken, otp }
5. Si PIN configuré → redirect /auth/pin → POST /auth/pin/verify
6. Réponse finale : { accessToken, user }
   + cookie httpOnly refresh_token posé par le backend
7. useAuthStore.setAuth(accessToken, user) → redirect /dashboard

```

Module Stratégies IA — interface d'import PDF

La page `/strategies/import` permet d'uploader un document PDF décrivant une stratégie de trading. Le backend extrait le texte, le tronque à 15 000 caractères et le soumet à `claude-sonnet-4-6`.

TypeScript

```

// frontend-web/pages/strategies/import.tsx (simplifié)
const handleImport = async () => {
  const form = new FormData();
  form.append('file', selectedFile);
  form.append('name', strategyName);
  form.append('asset', selectedAsset); // ex. 'BTC'
  form.append('timeframe', timeframe); // ex. '1h'

  const { data } = await apiClient.post('/strategies/import', form, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });
  // data.rules → StrategyRules JSON extrait par Claude
  // data.signal → Signal BUY créé si ENTRY_SIGNAL détecté
  setExtractedRules(data.rules);
};

```

Résultat affiché : les `StrategyRules` JSON extraites par Claude sont rendues de manière lisible — conditions d'entrée/sortie, indicateurs, paramètres de risk management (stop_loss %, take_profit %, confidence_score).

Page des signaux — design long-only

TypeScript

```
// frontend-web/pages/signals/index.tsx
// Un signal BUY OPEN = position active
// Un signal BUY CLOSED = trade terminé (exit_price renseigné)
// Il n'existe pas de signal SELL distinct dans la BDD

const statusColor = (signal: Signal) => {
  if (signal.status === 'OPEN') return 'text-green-400'; // #10b981
  if (signal.status === 'CLOSED') {
    const profitable = signal.exit_price > signal.entry_price;
    return profitable ? 'text-green-400' : 'text-red-400';
  }
  return 'text-slate-400';
};
```

La confiance (0–100 %) est affichée comme barre de progression amber. Les patterns détectés (`signal.patterns` JSON array) et les valeurs d'indicateurs (`signal.indicators` JSON object) sont dépliables.

Page Marchés — données CoinGecko en temps réel

Architecture proxy Next.js → CoinGecko

Pour éviter l'exposition de la clé API côté client et permettre le cache serveur, les appels CoinGecko passent par 3 routes API Next.js (`/api/coingecko/*`) qui proxifient les requêtes vers `api.coingecko.com/api/v3`.

TypeScript

```
// frontend-web/pages/api/coingecko/markets.ts
export default async function handler(req, res) {
  const response = await fetch(
    'https://api.coingecko.com/api/v3/coins/markets' +
    '?vs_currency=usd&order=market_cap_desc&per_page=20&sparkline=true',
    { headers: { 'x-cg-demo-api-key': process.env.COINGECKO_API_KEY } }
  );
  const data = await response.json();
  res.setHeader('Cache-Control', 's-maxage=60'); // cache CDN 60 s
  res.json(data);
}
```

Le backend dispose lui aussi de son propre cache Redis anti-429 (double clé fraîche + stale 1 h) — détaillé dans la section Architecture logicielle.

Graphique OHLCV — lightweight-charts 4.2.3

TypeScript

```
// frontend-web/pages/markets/[id].tsx (simplifié)
import { createChart, CandlestickSeries, LineSeries } from 'lightweight-charts';

useEffect(() => {
  const chart = createChart(chartRef.current, {
    layout: { background: { color: '#0f172a' }, textColor: '#94a3b8' },
    grid: { vertLines: { color: '#1e293b' }, horzLines: { color: '#1e293b' } },
    width: chartRef.current.clientWidth,
    height: 400,
  });
```

```

});

if (ohlcMode === 'candle') {
  // Mode chandelier — données depuis /markets/:id/ohlc (endpoint /ohlc CoinGecko)
  const series = chart.addSeries(CandlestickSeries, {
    upColor: '#10b981', downColor: '#ef4444',
    borderUpColor: '#10b981', borderDownColor: '#ef4444',
    wickUpColor: '#10b981', wickDownColor: '#ef4444',
  });
  series.setData(ohlcData); // [{ time, open, high, low, close }]
} else {
  // Fallback ligne — si /ohlc CoinGecko échoue ou données insuffisantes
  const series = chart.addSeries(LineSeries, { color: '#f59e0b', lineWidth: 2 });
  series.setData(lineData); // [{ time, value }]
}

return () => chart.remove();
}, [ohlcData, ohlcMode]);

```

Logique de fallback : `MarketsService.getOhlc()` tente d'abord l'endpoint `/coins/{id}/ohlc` (chandeliers natifs). Si CoinGecko retourne une erreur ou des données insuffisantes, il bascule sur `/coins/{id}/market_chart` (série de prix) — le frontend adapte l'affichage de chandelier à ligne sans action utilisateur.

Sélecteur de période

Les périodes disponibles (`1j`, `7j`, `30j`, `90j`) mappent sur le paramètre `days` de l'endpoint `/markets/:id/ohlc?days=N`. Le TTL Redis s'adapte :

Période	Paramètre `days`	TTL cache Redis
1 jour	1	60 s
7 jours	7	180 s
30 jours	30	600 s
90 jours	90	1 800 s

Module Formation (LMS intégré)

Modèle de données

```

Course (1) ■■■■ (N) Lesson
  ■
  ■■■■ UserProgress (N) ■■ (1) User
    UNIQUE [userId, lessonId]

```

Un `Course` possède un niveau (`DEBUTANT` | `INTERMEDIAIRE` | `AVANCE` | `EXPERT`), une catégorie (ex. `analyse_technique`, `gestion_du_risque`) et un ordre d'affichage. Chaque `Lesson` a un type : `VIDEO` (YouTube embed), `ARTICLE` (contenu Markdown) ou `QUIZ`.

Page catalogue `/formation`

TypeScript

```

// frontend-web/pages/formation/index.tsx
// Affichage : cards par cours avec progression Prisma UserProgress

```

```
const { data: courses } = await apiClient.get('/formation/courses');
// Chaque cours : { id, title, level, category, duration, totalLessons,
//                  completedLessons, progressPercent }
```

Les cours sont filtrables par niveau et catégorie. La progression (% de leçons complétées) est calculée côté backend par **FormationService** en comptant les **UserProgress.completed === true** pour ce cours et cet utilisateur.

Lecteur de leçon `/formation/[courseId]/[lessonId]`

TypeScript

```
// frontend-web/pages/formation/[courseId]/[lessonId].tsx
const lesson = await apiClient.get(`/formation/lessons/${lessonId}`);

// Type VIDEO : embed YouTube
if (lesson.type === 'VIDEO') {
  return <iframe src={`https://www.youtube.com/embed/${extractYouTubeId(lesson.videoUrl)}`}
    allow="accelerometer; autoplay; encrypted-media" allowFullScreen />;
}
// Type ARTICLE : rendu Markdown
if (lesson.type === 'ARTICLE') {
  return <ReactMarkdown>{lesson.content}</ReactMarkdown>;
}
// Type QUIZ : questions/réponses depuis lesson.content (JSON)
```

Marquage de complétion

TypeScript

```
const markComplete = async () => {
  await apiClient.post('/formation/progress', {
    lessonId,
    courseId,
    completed: true,
    score: quizScore ?? null,
  });
  // Contrainte UNIQUE [userId, lessonId] → upsert côté Prisma, pas de doublons
};
```

Sécurité en arrière-plan

Résumé des protections côté client

Risque	Mécanisme
Vol du token JWT	Access token en mémoire Zustand (pas de localStorage)
Accès aux pages protégées	getServerSideProps vérifie le cookie refresh_token
Requêtes non authentifiées	Intercepteur Axios injecte le token à chaque requête
Token expiré (15 min)	Intercepteur Axios déclenche POST /auth/refresh silencieux sur 401
Exposition clé CoinGecko	Proxy Next.js /api/coingecko/* — clé uniquement côté serveur
XSS	Toutes les données API rendues via JSX React (échappement automatique)

Risque	Mécanisme
CSRF	Token JWT en header (non forçable cross-origin) + cookie <code>SameSite: strict</code>

Gestion de la déconnexion

TypeScript

```
// frontend-web/pages/profile/index.tsx
const logout = async () => {
  await apiClient.post('/auth/logout'); // révoque refresh:{jti} dans Redis
  useAuthStore.getState().clearAuth(); // efface l'access token de la mémoire
  // Le cookie httpOnly est effacé par le backend (Set-Cookie: refresh_token=; Max-Age=0)
  router.push('/login');
};
```

La révocation côté Redis garantit que le refresh token est invalide immédiatement, même si l'utilisateur copie le cookie avant la déconnexion.

10. Application Mobile

Vue d'ensemble

L'application mobile Alvio est développée avec **React Native 0.72.3** et **Expo 49**, offrant une version compagnon de la plateforme web pour les utilisateurs qui souhaitent suivre leurs signaux de trading et simuler des stratégies DCA depuis leur smartphone.

Rôle dans l'écosystème Alvio

L'application mobile consomme la même API NestJS que le frontend web (**broker-ia-backend** sur le port 3001). Elle ne dispose pas d'un backend dédié — tous les calculs, la détection de patterns et l'accès aux données de marché passent par les mêmes endpoints REST.

Choix technologiques

Technologie	Version	Justification
React Native	0.72.3	Codebase JS/TS partageable avec le frontend web (Axios, logique métier)
Expo	49.0.0	Toolchain unifiée (build, test, OTA updates) sans config native Xcode/Gradle
@react-navigation/stack	^6.3.16	Navigation par pile — gestion du flux d'authentification
@react-navigation/bottom-tabs	^6.5.8	Navigation principale à 4 onglets inférieurs
Axios	^1.4.0	Client HTTP — même configuration d'intercepteurs que le web
AsyncStorage	^1.17.12	Persistance locale du token JWT entre les sessions
ionicons	^7.2.1	Icônes natives (compatibles iOS et Android)

Périmètre fonctionnel

L'application mobile couvre **5 écrans** fonctionnels. L'écran Markets (consultation des prix CoinGecko en temps réel avec graphiques OHLCV) n'est pas implémenté dans la version actuelle — il constitue la principale perspective d'évolution de l'application mobile.

Navigation — Structure réelle

TypeScript

```
// mobile/src/navigation/RootNavigator.tsx
// Deux navigateurs imbriqués : Stack (auth) + Bottom Tabs (app connectée)

const RootNavigator = () => {
  const { isAuthenticated } = useAuthStore();
```

Les 5 écrans réels

Écran 1 — LoginScreen

mobile/src/screens/auth/LoginScreen.tsx

Formulaire email/password. Sur succès, le backend renvoie un `accessToken` et pose le cookie `httpOnly refresh_token` (géré par le navigateur WebView d'Expo). L'access token est persisté dans `AsyncStorage` pour survivre aux relances de l'application.

TypeScript

```

const { data } = await apiClient.post('/auth/login', { email, password });

if (data.preAuthToken) {
  // Flux MFA : redirection vers saisie OTP
  // (simplifié dans l'app mobile – l'OTP est saisi sur la même page)
  const otpCode = await promptOtp();
  const verified = await apiClient.post('/auth/2fa/verify', {
    preAuthToken: data.preAuthToken,
    otp: otpCode,
  });
  // Persistance du token en local
  await AsyncStorage.setItem('access_token', verified.data.accessToken);
  setAuth(verified.data.accessToken, verified.data.user);
}
} catch (err) {
  Alert.alert('Erreur', err.response?.data?.message ?? 'Connexion échouée');
} finally {
  setLoading(false);
}
};

return (
  <View style={styles.container}>
    /* Logo Alvio – triangle + ligne ascendante */
    <AlvioLogo color="#f59e8b" size={64} />
    <Text style={styles.title}>Alvio</Text>

    <TextInput style={styles.input} placeholder="Email"
      value={email} onChangeText={setEmail} keyboardType="email-address"
      autoCapitalize="none" placeholderTextColor="#94a3b8" />
    <TextInput style={styles.input} placeholder="Mot de passe"
      value={password} onChangeText={setPassword} secureTextEntry />

    <TouchableOpacity style={styles.btn} onPress={handleLogin} disabled={loading}>
      {loading ? <ActivityIndicator color="#0f172a" />
        : <Text style={styles.btnText}>Se connecter</Text>}
    </TouchableOpacity>
  </View>
);
};

```

Écran 2 — DashboardScreen

`mobile/src/screens/dashboard/DashboardScreen.tsx`

Vue d'ensemble : signaux récents (5 derniers), statistiques rapides du mois en cours et accès rapide aux autres écrans.

TypeScript

```

// mobile/src/screens/dashboard/DashboardScreen.tsx
const DashboardScreen = () => {
  const [signals, setSignals] = useState([]);
  const [stats, setStats] = useState(null);
  const today = new Date();

  useEffect(() => {
    const load = async () => {
      const [sigRes, repRes] = await Promise.all([
        apiClient.get('/signals/recent'), // 5 derniers signaux
        apiClient.get(`/reports/${today.getFullYear()}/${today.getMonth() + 1}`),
      ]);
      setSignals(sigRes.data);
      setStats(repRes.data);
    };
  });
};

```

```

    load();
  }, []);

  return (
    <ScrollView style={styles.screen}>
      <Text style={styles.heading}>Bonjour 📈</Text>

      {/* Stats du mois */}
      {stats && (
        <View style={styles.statsRow}>
          <StatCard label="Signaux" value={stats.total_signals} color="#f59e0b" />
          <StatCard label="Win rate" value={` ${stats.win_rate}%` } color="#10b981" />
          <StatCard label="P&L est." value={` ${stats.total_pnl_estimate} $` }
            color={stats.total_pnl_estimate >= 0 ? '#10b981' : '#ef4444'} />
        </View>
      )}

      {/* Liste des 5 signaux récents – design long-only : direction toujours BUY */}
      <Text style={styles.sectionTitle}>Signaux récents</Text>
      {signals.map((s) => (
        <SignalCard key={s.id}
          asset={s.asset}
          status={s.status} // 'OPEN' ou 'CLOSED'
          confidence={s.confidence} // 0-100 %
          entryPrice={s.entry_price}
          exitPrice={s.exit_price} // null si OPEN
        />
      ))}
    </ScrollView>
  );
};

```

Note de design : les signaux sont toujours de direction **BUY** (long-only). **SignalCard** affiche **OPEN** en vert (#10b981) et **CLOSED** en amber (#f59e0b) si profitable, en rouge (#ef4444) si en perte.

Écran 3 — SignalsScreen

`mobile/src/screens/signals/SignalsScreen.tsx`

Liste paginée des 50 derniers signaux avec filtres par statut (OPEN / CLOSED) et par actif.

TypeScript

```

// mobile/src/screens/signals/SignalsScreen.tsx
const SignalsScreen = () => {
  const [signals, setSignals] = useState([]);
  const [filter, setFilter] = useState<'ALL' | 'OPEN' | 'CLOSED'>('ALL');

  useEffect(() => {
    apiClient.get('/signals').then(({ data }) => setSignals(data));
  }, []);

  const filtered = signals.filter((s) =>
    filter === 'ALL' ? true : s.status === filter
  );

  return (
    <View style={styles.screen}>
      {/* Filtres */}
      <View style={styles.filterRow}>
        {['ALL', 'OPEN', 'CLOSED'].map((f) => (
          <TouchableOpacity key={f} style={styles.filterBtn,
            filter === f && styles.filterActive} onPress={() => setFilter(f as any)}>
            <Text style={filter === f ? styles.filterTextActive : styles.filterText}>{f}</Text>
          </TouchableOpacity>
        ))}
      </View>

      <List>
        {filtered.map((s) => (
          <SignalCard key={s.id}
            asset={s.asset}
            status={s.status}
            confidence={s.confidence}
            entryPrice={s.entry_price}
            exitPrice={s.exit_price}
          />
        ))}
      </List>
    </View>
  );
};

```

```

        </TouchableOpacity>;
    ))}
    </View>;

    <FlatList
      data={filtered}
      keyExtractor={({item}) => item.id}
      renderItem={({ item }) => (
        <View style={styles.signalRow}>
          <Text style={styles.asset}>{item.asset}</Text>
          { /* Barre de confiance */ }
          <View style={styles.confBar}>
            <View style={[styles.confFill, { width: `${item.confidence}%`,
              backgroundColor: item.confidence > 75 ? '#18b981' : '#f59e0b' }]} />
            </View>
            <Text style={styles.conf}>{item.confidence.toFixed(1)}%</Text>
            <Text style={item.status === 'OPEN' ? styles.open : styles.closed}>
              {item.status}
            </Text>
          </View>
        </View>
      )}
    />
  </View>;
);
};

```

Écran 4 — SimulatorScreen

`mobile/src/screens/simulator/SimulatorScreen.tsx`

Formulaire de simulation DCA : capital initial, apport mensuel, durée, taux de rendement annuel, mode (**fixed** ou **monte_carlo**). Les résultats sont affichés immédiatement après le calcul.

TypeScript

```

// mobile/src/screens/simulator/SimulatorScreen.tsx
const SimulatorScreen = () => {
  const [form, setForm] = useState({
    initialAmount: '1000',
    monthlyAmount: '200',
    months: '24',
    annualReturn: '12',
    volatility: '5',
    mode: 'fixed' as 'fixed' | 'monte_carlo',
  });
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);

  const simulate = async () => {
    setLoading(true);
    try {
      const { data } = await apiClient.post('/simulator/dca', {
        asset: 'BTC',
        initialAmount: parseFloat(form.initialAmount),
        monthlyAmount: parseFloat(form.monthlyAmount),
        months: parseInt(form.months),
        annualReturn: parseFloat(form.annualReturn) / 100,
        volatility: parseFloat(form.volatility) / 100,
        mode: form.mode,
      });
      setResult(data.result);
    } finally {
      setLoading(false);
    }
  };
};

```

```

return (
  <ScrollView style={styles.screen}>
    <Text style={styles.heading}>Simulateur DCA</Text>

    <InputField label="Capital initial ($)" value={form.initialAmount}
      onChangeText={(v) => setForm({ ...form, initialAmount: v })}
      keyboardType="numeric" />
    <InputField label="Apport mensuel ($)" value={form.monthlyAmount}
      onChangeText={(v) => setForm({ ...form, monthlyAmount: v })}
      keyboardType="numeric" />
    <InputField label="Durée (mois)" value={form.months}
      onChangeText={(v) => setForm({ ...form, months: v })}
      keyboardType="numeric" />
    <InputField label="Rendement annuel (%)" value={form.annualReturn}
      onChangeText={(v) => setForm({ ...form, annualReturn: v })}
      keyboardType="numeric" />

    /* Mode de simulation */
    <View style={styles.modeRow}>
      {[ 'fixed', 'monte_carlo' ] as const}.map((m) => (
        <TouchableOpacity key={m} style={[styles.modeBtn,
          form.mode === m && styles.modeBtnActive]}
          onPress={() => setForm({ ...form, mode: m })}>
          <Text style={styles.modeText}>{m === 'fixed' ? 'Fixe' : 'Monte Carlo'}</Text>
        </TouchableOpacity>
      ))
    </View>

    <TouchableOpacity style={styles.btn} onPress={simulate} disabled={loading}>
      {loading ? <ActivityIndicator color="#0f172a" />
        : <Text style={styles.btnText}>Simuler</Text>}
    </TouchableOpacity>

    /* Résultat */
    {result && (
      <View style={styles.resultCard}>
        <ResultRow label="Capital investi" value={`$${result.totalInvested} $`} />
        <ResultRow label="Valeur finale" value={`$${result.finalBalance} $`}
          color="#1b981" />
        <ResultRow label="Gains totaux" value={`$${result.totalGains} $`}
          color={result.totalGains >= 0 ? '#1b981' : '#ef4444'} />
        <ResultRow label="ROI" value={`$${result.roi.toFixed(1)} %`}
          color="#f59e8b" />
      </View>
    )}
  </ScrollView>
);

```

Le mode `monte_carlo` applique un bruit gaussien (algorithme Box-Muller) côté backend pour simuler la volatilité du marché — chaque exécution produit un résultat légèrement différent.

Écran 5 — ProfileScreen

`mobile/src/screens/profile/ProfileScreen.tsx`

Affiche les informations du profil de l'utilisateur (username, email, préférences de trading) et propose la déconnexion.

TypeScript

```

// mobile/src/screens/profile/ProfileScreen.tsx
const ProfileScreen = () => {
  const { user, clearAuth } = useAuthStore();

```

```

const logout = async () => {
  try {
    await apiClient.post('/auth/logout'); // révoque refresh:{jti} dans Redis
  } finally {
    await AsyncStorage.removeItem('access_token'); // supprime le token local
    clearAuth(); // vide le store Zustand
  }
};

return (
  <ScrollView style={styles.screen}>
    <View style={styles.avatarZone}>
      {/* Initiales dans un cercle amber */}
      <View style={styles.avatar}>
        <Text style={styles.avatarText}>
          {user?.username?.charAt(0).toUpperCase() ?? '?'}
        </Text>
      </View>
      <Text style={styles.username}>{user?.username}</Text>
      <Text style={styles.email}>{user?.email}</Text>
    </View>

    <View style={styles.section}>
      <Text style={styles.label}>Préférence de trading</Text>
      <Text style={styles.value}>{user?.trading_preference ?? 'moderate'}</Text>
    </View>

    <TouchableOpacity style={styles.logoutBtn} onPress={logout}>
      <Icons name="log-out-outline" size={20} color="#ef4444" />
      <Text style={styles.logoutText}>Se déconnecter</Text>
    </TouchableOpacity>
  </ScrollView>
);
};

```

Intégration backend

Configuration du client HTTP

TypeScript

```

// mobile/src/services/apiClient.ts
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const apiClient = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL ?? 'http://localhost:3001',
  withCredentials: true, // transmet les cookies (refresh_token httpOnly)
  timeout: 10_000,
});

// Injection du token JWT à chaque requête
apiClient.interceptors.request.use(async (config) => {
  const token = await AsyncStorage.getItem('access_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Refresh automatique sur 401
apiClient.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {

```

```
error.config._retry = true;
try {
  const { data } = await axios.post(
    `${process.env.EXPO_PUBLIC_API_URL}/auth/refresh`,
    {},
    { withCredentials: true }
  );
  await AsyncStorage.setItem('access_token', data.accessToken);
  error.config.headers.Authorization = `Bearer ${data.accessToken}`;
  return apiClient(error.config);
} catch {
  // Refresh échoué → déconnexion forcée
  await AsyncStorage.removeItem('access_token');
  useAuthStore.getState().clearAuth();
}
}
return Promise.reject(error);
});
```

Endpoints consommés par l'application mobile

Écran	Endpoint NestJS	Description
LoginScreen	POST /auth/login	Login → preAuthToken
LoginScreen	POST /auth/2fa/verify	Vérification OTP email
LoginScreen	POST /auth/refresh	Refresh du token JWT
DashboardScreen	GET /signals/recent	5 signaux les plus récents
DashboardScreen	GET /reports/:year/:month	Stats du mois en cours
SignalsScreen	GET /signals	50 derniers signaux
SimulatorScreen	POST /simulator/dca	Simulation DCA (fixed/monte_carlo)
SimulatorScreen	GET /simulator/history	Historique des simulations
ProfileScreen	GET /auth/profile	Données utilisateur
ProfileScreen	POST /auth/logout	Révocation token Redis

Perspectives d'évolution

Les fonctionnalités suivantes ne sont pas implémentées dans la version mobile actuelle et constituent des axes de développement futurs :

Fonctionnalité	Justification du report
Écran Markets (prix CoinGecko + OHLCV)	Implémentation complexe (graphiques natifs) — réservée à une prochaine itération
Module Formation (LMS mobile)	Lecture vidéo YouTube et contenu Markdown nécessitent des packages natifs supplémentaires
Notifications push	Dépend d'Expo Notifications + configuration côté backend (webhook signal → push)
Authentification GitHub OAuth	Flux OAuth nécessite un in-app browser natif (expo-web-browser)

Fonctionnalité	Justification du report
TOTP / WebAuthn mobile	Biométrie native disponible via expo-local-authentication

11. Module IA / ML

Pipeline IA global

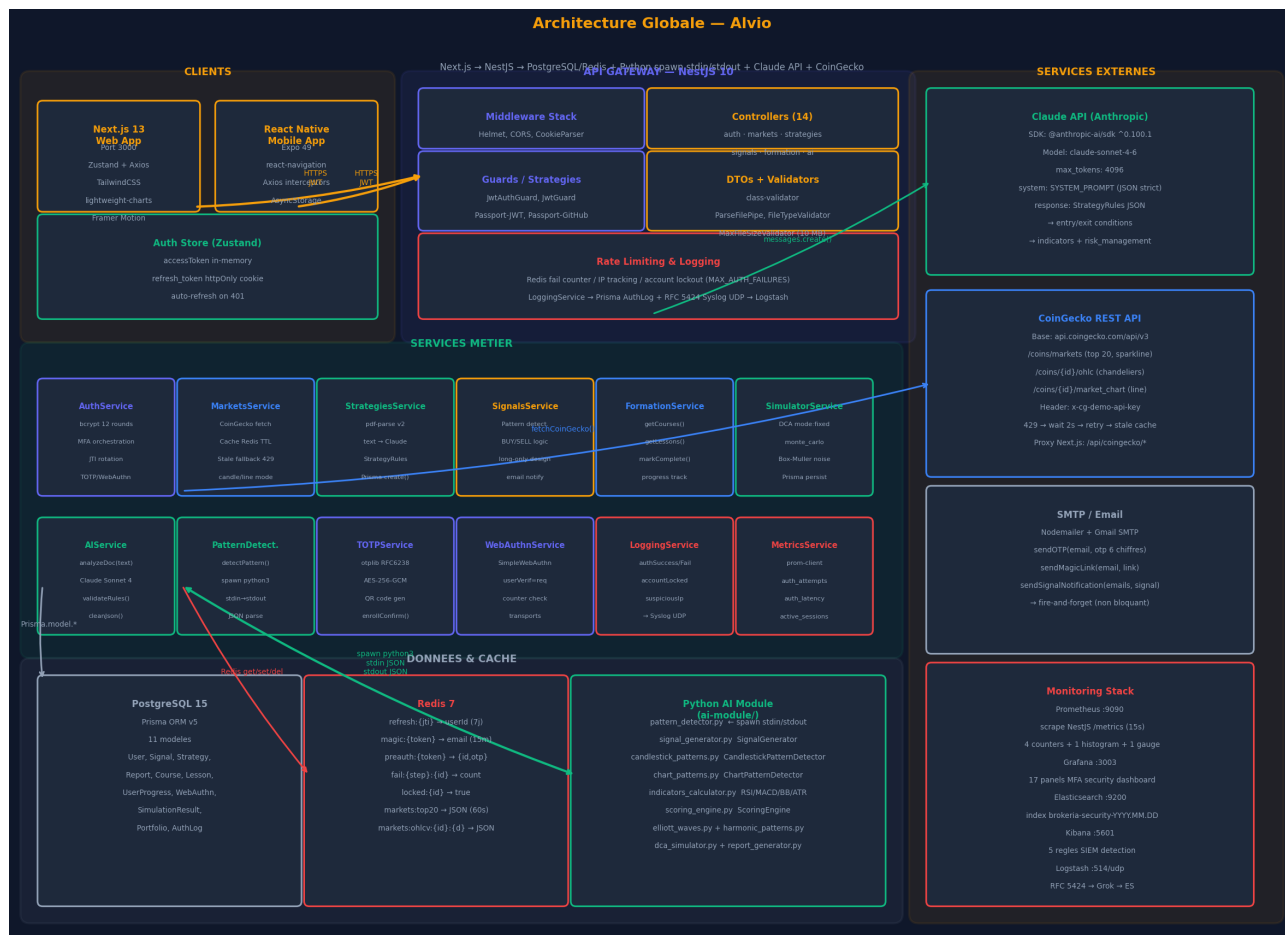


Figure — Architecture IA Alvio (architecture.png)

Le module IA d'Alvio implémente une chaîne de traitement en trois étapes successives, de l'import du document de stratégie jusqu'à la génération du signal de trading :

Utilisateur

- 1. Upload fichier PDF/TXT/MD

Multer (memoryStorage, 10 MB max)

- 2. Extraction texte brut

pdf-parse v2 → PDFParse.getText()
(tronqué à 15 000 caractères)

- 3. Analyse LLM

Anthropic Claude API (claude-sonnet-4-6)

- System prompt → JSON strict

```

    ■ Réponse : StrategyRules { entry_conditions, exit_conditions,
    ■                               indicators, risk_management, confidence_score }
    ▼
    Prisma.strategy.create({ code: JSON.stringify(rules) })
    ■
    ■ 4. Détection de patterns (immédiate ou scheduler 15 min)
    ▼
    PatternDetectionService
    ■ → MarketsService.getOhlcv() → données CoinGecko
    ■ → spawn('python3', ['pattern_detector.py'])
    ■     stdin : JSON { ohlcv[], strategy_rules, asset, timeframe }
    ■     stdout : JSON PatternDetectionResult { global_status, signals,
    ■                                             patterns, indicators }
    ▼
    SignalsService.generateSignal()
    ■ ENTRY_SIGNAL → Prisma.signal.create({ direction:'BUY', status:'OPEN' })
    ■ EXIT_SIGNAL → Prisma.signal.update({ status:'CLOSED', exit_price })
    ▼
    EmailService.sendSignalNotification() (fire-and-forget)

```

Import et parsing PDF

Réception du fichier — Multer

TypeScript

```

// src/strategies/strategies.controller.ts
@Post('import')
@UseGuards(JwtAuthGuard)
@UseInterceptors(
  FileInterceptor('file', {
    storage: memoryStorage(), // fichier conservé en Buffer RAM, jamais écrit sur disque
  })
)
async importFromDocument(
  @UploadedFile()
  new ParseFilePipe({
    validators: [
      new MaxFileSizeValidator({ maxSize: 10 * 1024 * 1024 }), // 10 MB
      new FileTypeValidator({ fileType: /(pdf|text|markdown)/ }),
    ],
  }) file: Express.Multer.File,
  @Body() dto: ImportStrategyDto,
  @GetUser('sub') userId: string,
) {
  return this.strategiesService.importFromDocument(file, dto, userId);
}

```

Extraction du texte — pdf-parse v2

La version 2 de **pdf-parse** rompt avec l'API v1 : la fonction directe est remplacée par une classe **PDFParse** à instancier avec un objet `{ data: Buffer }`.

TypeScript

```

// src/strategies/strategies.service.ts
private async extractText(file: Express.Multer.File): Promise<string> {
  // Import dynamique nécessaire — pdf-parse v2 n'a pas d'export ES module standard
  const { PDFParse } = require('pdf-parse') as {
    PDFParse: new (opts: { data: Buffer }) => { getText(): Promise<string> };
  };
}

```

```
const parser = new PDFParse({ data: file.buffer });
const text = await parser.getText();

// Troncature avant envoi à Claude – limite de tokens du prompt
const MAX_TEXT_LENGTH = 15_000;
return text.slice(0, MAX_TEXT_LENGTH);
}
```

Pourquoi 15 000 caractères ? Le **SYSTEM_PROMPT** de Claude consomme déjà plusieurs centaines de tokens. Tronquer le texte à ~15 000 caractères garantit de rester dans la fenêtre de **max_tokens** : 4096 définie pour la réponse, sans risquer un dépassement côté entrée.

Strategy Engine IA — Appel Claude

Modèle et constante

TypeScript

```
// src/ai/ai.service.ts — ligne 14
const CLAUDE_MODEL = 'claude-sonnet-4-6'; // modèle fixé, jamais modifié à l'exécution
```

Prompt système

Le **SYSTEM_PROMPT** impose à Claude de répondre **exclusivement** en JSON, sans texte explicatif, sans balises Markdown, selon un schéma fixé :

TypeScript

```
// src/ai/ai.service.ts
const SYSTEM_PROMPT = `
Tu es un expert en analyse technique et en trading algorithmique.
Ton rôle est d'analyser les documents de stratégie de trading et d'en extraire
les règles de manière structurée.

IMPORTANT : Réponds UNIQUEMENT avec un objet JSON valide, sans texte supplémentaire,
sans balises markdown. Structure exacte requise :
{
  "name": "string",
  "description": "string",
  "asset": "string",
  "timeframe": "string",
  "entry_conditions": ["condition1", "condition2"],
  "exit_conditions": ["condition1", "condition2"],
  "indicators": [{ "name": "string", "params": {} }],
  "risk_management": {
    "stop_loss_percent": number,
    "take_profit_percent": number,
    "max_position_size": number
  },
  "confidence_score": number // 0 à 100
}
`;
```

Appel API et traitement de la réponse

TypeScript

```
// src/ai/ai.service.ts
```

```

async analyzeStrategyDocument(text: string): Promise<StrategyRules> {
  const client = new Anthropic({ apiKey: this.config.get('ANTHROPIC_API_KEY') });

  const response = await client.messages.create({
    model:      CLAUDE_MODEL,      // 'claude-sonnet-4-6'
    max_tokens: 4096,
    system:     SYSTEM_PROMPT,
    messages:   [{ role: 'user', content: text }],
  });

  // Extraction du texte brut de la réponse (type TextBlock)
  const raw = (response.content[0] as TextBlock).text;

  // Nettoyage : Claude peut encadrer le JSON de balises ```json ... ```
  const cleaned = this.cleanJsonResponse(raw);

  // Parse et validation de structure
  const parsed = JSON.parse(cleaned);
  if (!this.validateStrategyRules(parsed)) {
    throw new BadRequestException('Structure StrategyRules invalide retournée par Claude');
  }

  // Normalisation du confidence_score entre 0 et 100
  return {
    ...parsed,
    confidence_score: Math.min(100, Math.max(0, Number(parsed.confidence_score))),
  };
}

private cleanJsonResponse(raw: string): string {
  // Retire les blocs ```json ... ``` si Claude les ajoute malgré les instructions
  return raw
    .replace(/```json\s*/i, '')
    .replace(/```\s*/, '')
    .replace(/\s*```$/, '')
    .trim();
}

private validateStrategyRules(obj: unknown): obj is StrategyRules {
  if (typeof obj !== 'object' || obj === null) return false;
  const r = obj as Record<string, unknown>;
  return {
    typeof r.name           === 'string' &&
    Array.isArray(r.entry_conditions) &&
    Array.isArray(r.exit_conditions) &&
    typeof r.risk_management === 'object' &&
    typeof r.confidence_score === 'number'
  };
}

```

Bridge Python — `spawn` stdin/stdout

Choix de `spawn` vs `exec`

Le bridge utilise `child_process.spawn` avec `stdio`: `['pipe', 'pipe', 'pipe']` plutôt qu'`exec` pour deux raisons :

1. Flux streaming : `spawn` expose les flux `stdin/stdout` en temps réel —

les données OHLCV (potentiellement volumineuses) sont écrites progressivement sans bufferiser toute la sortie en mémoire comme le ferait `exec`.

2. Absence de limite de buffer : **exec** a une limite de taille sur **stdout** (**maxBuffer** par défaut ~1 Mo) — les résultats d'analyse multi-patterns peuvent dépasser cette limite.

Implémentation du bridge

TypeScript

```
// src/patterns/pattern-detection.service.ts

// Mapping asset → identifiant CoinGecko
private assetToCoinId(asset: string): string {
  const map: Record<string, string> = {
    BTC: 'bitcoin', ETH: 'ethereum', SOL: 'solana',
    BNB: 'binancecoin', XRP: 'ripple', ADA: 'cardano',
    AVAX: 'avalanche-2', DOT: 'polkadot', MATIC: 'matic-network',
  };
  return map[asset.toUpperCase()] ?? asset.toLowerCase();
}

// Mapping timeframe → fenêtre de données en jours
private timeframeToDays(timeframe: string): number {
  const map: Record<string, number> = {
    '15m': 7, '30m': 7, // données intraday → 7 j
    '1h': 14, // 1 heure → 14 j
    '4h': 30, // 4 heures → 30 j
    '1d': 90, '1w': 90, // quotidien/hebdo → 90 j
  };
  return map[timeframe] ?? 30;
}

async detectPattern(
  strategyId: string,
  asset: string,
  timeframe: string = '1h'
): Promise<PatternDetectionResult> {
  // 1. Récupération de la stratégie et de ses règles JSON
  const strategy = await this.prisma.strategy.findUnique({ where: { id: strategyId } });
  const rules: StrategyRules = JSON.parse(strategy.code);

  // 2. Données OHLCV depuis CoinGecko (via MarketsService, avec cache Redis)
  const coinId = this.assetToCoinId(asset);
  const days = this.timeframeToDays(timeframe);
  const ohlcv = await this.marketsService.getOhlcv(coinId, days);

  // 3. Construction de l'entrée JSON pour le script Python
  const input: PatternDetectionInput = {
    ohlcv: this.transformOhlcv(ohlcv), // [{ time, open, high, low, close, volume }]
    strategy_rules: rules,
    asset,
    timeframe,
  };

  // 4. Appel du bridge Python via spawn
  return this.runPatternDetector(input);
}

private runPatternDetector(input: PatternDetectionInput): Promise<PatternDetectionResult> {
  return new Promise((resolve, reject) => {
    // spawn avec stdio pipes — jamais exec
    const proc = spawn('python3', ['/app/ai-module/pattern_detector.py'], {
      stdio: ['pipe', 'pipe', 'pipe'],
    });

    let stdout = '';
```

```

let stderr = '';

proc.stdout.on('data', (chunk) => { stdout += chunk.toString(); });
proc.stderr.on('data', (chunk) => { stderr += chunk.toString(); });

proc.on('close', (code) => {
  if (code !== 0) {
    return reject(new Error(`pattern_detector.py exited with code ${code}: ${stderr}`));
  }
  try {
    resolve(JSON.parse(stdout) as PatternDetectionResult);
  } catch {
    reject(new Error(`JSON parse error from Python output: ${stdout}`));
  }
});

// Envoi des données d'entrée via stdin
proc.stdin.write(JSON.stringify(input), 'utf-8');
proc.stdin.end(); // fermeture du flux → le script Python peut commencer la lecture
});
}

```

Format des données échangées

Entrée (NestJS → Python, via stdin) :

JSON

```

{
  "ohlcv": [
    { "time": 1717200000, "open": 67420.5, "high": 68100.0,
      "low": 67100.0, "close": 67850.0, "volume": 1842.3 }
  ],
  "strategy_rules": {
    "entry_conditions": ["RSI < 30", "EMA20 > EMA50"],
    "exit_conditions": ["RSI > 70"],
    "indicators": [
      { "name": "RSI", "params": { "period": 14 } },
      { "name": "EMA", "params": { "period": 20 } }
    ],
    "risk_management": { "stop_loss_percent": 2, "take_profit_percent": 6 }
  },
  "asset": "BTC",
  "timeframe": "1h"
}

```

Sortie (Python → NestJS, via stdout) :

JSON

```

{
  "global_status": "ENTRY_SIGNAL",
  "confidence_score": 87.3,
  "patterns": ["bullish_engulfing", "hammer"],
  "indicators": {
    "rsi": { "value": 28.4, "signal": "oversold" },
    "ema_20": { "value": 67200.0 },
    "ema_50": { "value": 66800.0, "signal": "bullish_cross" },
    "macd": { "value": 120.5, "signal": 85.2, "histogram": 35.3 }
  },
  "entry_price": 67850.0,
  "stop_loss": 66493.0,
  "take_profit": 71923.0
}

```

global_status prend l'une de trois valeurs : **ENTRY_SIGNAL**, **EXIT_SIGNAL** ou **NO_SIGNAL**.

Détection de patterns — module Python

Scripts implémentés

Le module Python ([ai-module/](#)) contient 15 scripts organisés en classes :

Script	Classe principale	Indicateurs/patterns
indicators_calculator.py	TechnicalIndicators	RSI (14), MACD (12/26/9), Stochastique (%K/%D), Bandes de Bollinger (20/2σ), ATR (14)
candlestick_patterns.py	CandlestickPatternDetector	Hammer, Shooting Star, Doji, Engulfing (haussier/baissier), Morning/Evening Star, Harami
chart_patterns.py	ChartPatternDetector	Head & Shoulders, Double Top/Bottom, Triangle (ascendant/descendant/symétrique), Wedge
elliott_waves.py	ElliottWave	Détection des vagues d'Elliott (1-2-3-4-5 impulsives + A-B-C correctives)
harmonic_patterns.py	HarmonicPatterns	Gartley, Butterfly, Bat, Crab (ratios de Fibonacci)
ichimoku_indicator.py	IchimokuIndicator	Tenkan-sen, Kijun-sen, Senkou A/B, Chikou Span, Kumo cloud
scoring_engine.py	ScoringEngine	Agrégation des signaux → confidence_score 0–100
signal_generator.py	SignalGenerator	Dataclass TradingSignal — encapsule le signal final
dca_simulator.py	DCASimulator	Calcul d'intérêt composé (fixed + Box-Muller)
nlp_rule_extractor.py	—	Extraction NLP des règles (redondant avec Claude — non utilisé activement)
pattern_detector.py	—	Point d'entrée du bridge — orchestre les modules ci-dessus

Stack Python :

```
pandas 2.0.3      → manipulation des séries temporelles OHLCV
numpy 1.24.3     → calculs vectoriels (FFT, Box-Muller, Fibonacci)
TA-Lib 0.4.27    → indicateurs techniques optimisés (C binding)
scikit-learn 1.3.0 → classification de patterns (SVM, RandomForest)
scipy 1.11.0     → analyse spectrale (ondes d'Elliott)
```

Extrait — calcul RSI avec TA-Lib

Python

```
# ai-module/indicators_calculator.py
import talib
import numpy as np

class TechnicalIndicators:
```

```
def calculate_rsi(self, close: np.ndarray, period: int = 14) -> np.ndarray:
    """RSI via TA-Lib (Wilder smoothing) – retourne un array de la même longueur."""
    return talib.RSI(close, timeperiod=period)

def calculate_macd(self, close: np.ndarray):
    """MACD (12,26,9) – retourne (macd, signal, histogram)."""
    return talib.MACD(close, fastperiod=12, slowperiod=26, signalperiod=9)

def calculate_bollinger_bands(self, close: np.ndarray, period: int = 20):
    """Bandes de Bollinger (20, 2σ) – retourne (upper, middle, lower)."""
    return talib.BBANDS(close, timeperiod=period, nbdevup=2, nbdevdn=2)

def calculate_atr(self, high, low, close, period: int = 14) -> np.ndarray:
    """Average True Range – mesure de volatilité."""
    return talib.ATR(high, low, close, timeperiod=period)
```

Extrait — scoring engine

Python

```
# ai-module/scoring_engine.py
class ScoringEngine:
    def compute_confidence(self, signals: list[dict]) -> float:
        """
        Agrège les signaux de différents détecteurs en un score 0-100.
        Chaque signal porte un poids selon la fiabilité du détecteur.
        """
        weights = {
            'candlestick': 0.25,
            'chart': 0.20,
            'indicator': 0.35,  # RSI/MACD → poids fort
            'ichimoku': 0.15,
            'harmonic': 0.05,
        }
        total_score = 0.0
        for sig in signals:
            w = weights.get(sig['type'], 0.1)
            total_score += sig['strength'] * w * 100

        return min(100.0, max(0.0, total_score))
```

Génération des signaux — design long-only

Décision de design assumée

Alvio adopte une architecture de signaux **long-only** (achat uniquement), cohérente avec les stratégies de trading au comptant sur les cryptomonnaies. Ce choix est délibéré : sur les marchés spot (sans levier ni vente à découvert), un trader individuel ne peut pas "shorter" un actif nativement.

Conséquence sur le modèle de données :

- Un signal **BUY** est créé à l'entrée (**ENTRY_SIGNAL**) → **status**: 'OPEN'
- La sortie (**EXIT_SIGNAL**) **met à jour** le signal BUY existant :

status: 'CLOSED', **exit_price**, **closedAt** renseignés

- Il n'existe **pas de signal SELL distinct** en base de données

Déduplication des entrées

TypeScript

```
// src/signals/signals.service.ts
async generateSignal(userId: string, dto: GenerateSignalDto): Promise<Signal | null> {
  const detection = await this.patternDetection.detectPattern(
    dto.strategyId, dto.asset, dto.timeframe
  );

  if (detection.global_status === 'ENTRY_SIGNAL') {
    // Vérification : pas de BUY OPEN déjà existant sur cet asset/stratégie
    const existing = await this.prisma.signal.findFirst({
      where: {
        strategyId: dto.strategyId,
        asset:      dto.asset,
        direction:  'BUY',
        status:     'OPEN',
      },
    });

    if (existing) {
      // Déduplication – on ne crée pas deux entrées simultanées sur le même actif
      return null;
    }

    // Création du signal d'entrée
    const signal = await this.prisma.signal.create({
      data: {
        userId,
        strategyId: dto.strategyId,
        asset:      dto.asset,
        timeframe:  dto.timeframe,
        direction:  'BUY',
        status:     'OPEN',
        entry_price: detection.entry_price,
        stop_loss:  detection.stop_loss,
        take_profit: detection.take_profit,
        // Stockage du score de confiance en pourcentage (0.00-100.00)
        confidence: Math.round(detection.confidence_score * 10_000) / 100,
        patterns:   JSON.stringify(detection.patterns),
        indicators: JSON.stringify(detection.indicators),
      },
    });

    // Notification email – fire-and-forget (non bloquant)
    this.email.sendSignalNotification([userId], signal).catch(() => {});
    return signal;
  }

  if (detection.global_status === 'EXIT_SIGNAL') {
    // Recherche du BUY OPEN correspondant
    const openSignal = await this.prisma.signal.findFirst({
      where: { strategyId: dto.strategyId, asset: dto.asset,
        direction: 'BUY', status: 'OPEN' },
    });

    if (!openSignal) return null; // aucune position à fermer

    // Clôture du signal – mise à jour (pas de création d'un SELL)
    return this.prisma.signal.update({
      where: { id: openSignal.id },
      data: {
        status: 'CLOSED',
        exit_price: detection.entry_price, // prix de marché au moment de l'EXIT
        closedAt: new Date(),
      },
    });
  }
}
```

```

    return null; // NO_SIGNAL — aucune action
  }

```

Cycle de vie d'un signal

```

PatternDetector retourne ENTRY_SIGNAL
  ▼
signal.create({ direction:'BUY', status:'OPEN', entry_price, stop_loss, take_profit })
  ▼
  [Temps qui passe — scheduler re-scanner toutes les 15 min]
  ▼
PatternDetector retourne EXIT_SIGNAL
  ▼
signal.update({ status:'CLOSED', exit_price, closedAt })
  ▼
ReportsService : inclut ce signal dans le calcul mensuel (closedAt dans le mois)

```

Scheduler automatique

TypeScript

```

// src/signals/signal-scheduler.service.ts
@Cron('* / 15 * * * *') // toutes les 15 minutes
async runScan(): Promise<void> {
  // Récupère toutes les stratégies actives de tous les utilisateurs
  const strategies = await this.prisma.strategy.findMany({
    where: { status: 'active' },
  });

  for (const strategy of strategies) {
    await this.signalsService.generateSignal(strategy.userId, {
      strategyId: strategy.id,
      asset:      strategy.asset,
      timeframe:  strategy.timeframe,
    }).catch(() => {}); // erreur isolée — pas d'interruption du scan global
  }
}

```

Rapports mensuels automatisés

ReportsService est entièrement implémenté. Il calcule les statistiques mensuelles à la demande (**GET /reports/:year/:month**) à partir des signaux Prisma — sans pré-agrégation ni tâche cron dédiée.

TypeScript

```

// src/reports/reports.service.ts (extrait — voir fichier complet)
async getMonthlyStats(userId: string, year: number, month: number): Promise<MonthlyStats> {
  const monthStart = new Date(year, month - 1, 1);
  const monthEnd   = new Date(year, month, 1);

  const allSignals: Signal[] = await this.prisma.signal.findMany({ where: { userId } });

  // buy_signals = entrées déclenchées ce mois (createdAt dans [monthStart, monthEnd])
  const buySignals = allSignals.filter(s => s.createdAt >= monthStart && s.createdAt < monthEnd);
  // sell_signals = sorties clôturées ce mois (closedAt dans [monthStart, monthEnd])
  const sellSignals = allSignals.filter(s => s.closedAt != null && s.closedAt >= monthStart && s.closedAt < monthEnd);
  // hold_signals = positions OPEN toujours ouvertes en fin de mois
  const holdSignals = allSignals.filter(s => s.status === 'OPEN' && s.createdAt < monthEnd);
}

```

```

// Win rate : % clôtures avec exit_price > entry_price (sur les sorties du mois)
const closed = sellSignals.filter(s => s.exit_price != null);
const winning = closed.filter(s => (s.exit_price as number) > s.entry_price);
const winRate = closed.length > 0
  ? Math.round((winning.length / closed.length) * 10_000) / 100
  : 0;

// P&L estimé : somme des (exit_price - entry_price) des clôtures du mois
const totalPnl = closed.reduce((sum, s) =>
  sum + ((s.exit_price as number) - s.entry_price), 0);

return { year, month, buy_signals: buySignals.length, sell_signals: sellSignals.length,
  hold_signals: holdSignals.length, win_rate: winRate,
  total_pnl_estimate: Math.round(totalPnl * 100) / 100, /* ... */ };
}

```

Limites et perspectives d'évolution

Fonctionnalités non encore implémentées

Deux fonctionnalités de l'**AIService** retournent actuellement des **données fictives hardcodées** et ne doivent pas être présentées comme fonctionnelles :

`backtestStrategy()` — Backtesting

TypeScript

```

// src/ai/ai.service.ts — lignes 262-278
async backtestStrategy(strategyId: string): Promise<BacktestResult> {
  // TODO : implémenter le vrai backtesting
  // Actuellement : retourne des données fictives pour éviter les erreurs d'API
  return {
    totalTrades: 42,
    winRate: 67.5,
    profitFactor: 1.8,
    maxDrawdown: 12.3,
    sharpeRatio: 1.4,
    totalReturn: 34.7,
    // Ces valeurs sont hardcodées – elles ne reflètent aucune donnée réelle
  };
}

```

Piste d'évolution : implémenter un backtesting réel en récupérant l'historique OHLCV CoinGecko sur 1 à 3 ans, en appliquant les **StrategyRules** signal par signal, et en calculant les métriques (drawdown, Sharpe, profit factor) depuis les trades simulés.

`getPatternsHistory()` — Historique des patterns

TypeScript

```

// src/ai/ai.service.ts — lignes 247-256
async getPatternsHistory(userId: string): Promise<PatternHistory[]> {
  // TODO : agréger l'historique depuis Prisma Signal.patterns (champ JSON)
  // Actuellement : retourne un tableau fictif
  return [
    { pattern: 'bullish_engulfing', count: 12, successRate: 75 },
    { pattern: 'rsi_oversold', count: 8, successRate: 62 },
    // Ces données sont hardcodées – elles ne proviennent pas de la base
  ];
}

```

Piste d'évolution : agréger le champ `Signal.patterns` (JSON array) de tous les signaux CLOSED de l'utilisateur, calculer le taux de réussite (`exit_price > entry_price`) par pattern, et retourner un vrai classement. Les données sont déjà présentes en base — il s'agit uniquement d'écrire la requête d'agrégation.

Autres perspectives

Limite actuelle	Évolution envisageable
Polling HTTP (scheduler 15 min)	WebSocket temps réel — push immédiat lors d'un ENTRY_SIGNAL
Pattern detector monolithique	Pipeline distribuée (Celery / Redis Queue) pour les scans parallèles multi-utilisateurs
<code>nlp_rule_extractor.py</code> non utilisé	Remplacement partiel de Claude par extraction NLP locale pour réduire les coûts API
Signaux long-only	Extension aux marchés avec levier (futures) — nécessite des signaux SELL réels
Modèle Claude fixé	Rendre le modèle configurable par utilisateur ou par type d'analyse

12. Tests — Unitaires & Intégration

Stratégie de test

La stratégie de test d'Alvio est **concentrée sur les couches critiques** : les services d'authentification, de sécurité et d'infrastructure MFA font l'objet de tests automatisés approfondis. Les autres modules (marchés, formation, simulateur) sont validés manuellement via des appels Postman/cURL pendant le développement.

Outils utilisés

Couche	Framework	Version	Rôle
Backend — tests unitaires	Jest	^29.5.0	Tests des services NestJS avec mocks
Backend — couverture	Jest <code>--coverage</code>	—	Seuil configuré à 70 % (branches, fonctions, lines, statements)
Frontend — tests composants	Vitest	^4.1.0	Tests unitaires des composants React
Frontend — rendu	@testing-library/react	^16.3.2	Assertions sur le DOM rendu
Python — tests unitaires	unittest (stdlib)	—	Tests des classes du module IA

***Note** : aucun test end-to-end (Cypress, Playwright) ni test d'intégration HTTP (Supertest) n'est présent dans le projet actuel. C'est une limite documentée et assumée — voir section Perspectives.*

Seuil de couverture Jest

JavaScript

```
// backend-code/jest.config.js (ou package.json, section "jest")
{
  "coverageThreshold": {
    "global": {
      "branches": 70,
      "functions": 70,
      "lines": 70,
      "statements": 70
    }
  }
}
```

Le seuil de 70 % est vérifié à chaque `npm run test:cov`. En dessous de ce seuil, la commande se termine en erreur — ce qui en fait un garde-fou minimal dans le workflow de développement.

Tests unitaires Backend (Jest — 5 fichiers)

Fichiers de tests existants

Fichier	Service testé	Aspects couverts
auth/auth.service.spec.ts	AuthService	register, login, logout, 2FA, PIN
logging/logging.service.spec.ts	LoggingService	authSuccess/Failure, émission Syslog
metrics/metrics.service.spec.ts	MetricsService	compteurs Prometheus, histogram
mfa/totp/totp.service.spec.ts	TOTPService	enrollInit/Confirm, chiffrement AES-256-GCM
mfa/webauthn/webauthn.service.spec.ts	WebAuthnService	registrationOptions, verify

Extrait 1 — `auth.service.spec.ts`

Les dépendances Prisma et Redis sont mockées — le test porte sur la logique du service, pas sur la base de données réelle.

TypeScript

Extrait 2 — `totp.service.spec.ts`

TypeScript

```

    // L'URL doit être compatible avec Google Authenticator
    expect(result.otpauthUrl).toMatch(/^otpauth:\:\/\/totp\/\//);
  });
});

describe('enrollConfirm()', () => {
  it('devrait activer TOTP si le code OTP est valide', async () => {
    const secret = authenticator.generateSecret();
    // Génération d'un code TOTP valide pour ce secret
    const validCode = authenticator.generate(secret);

    const result = await service.enrollConfirm('user_abc', secret, validCode);
    expect(result).toBe(true);
  });

  it('devrait rejeter si le code OTP est invalide', async () => {
    const secret = authenticator.generateSecret();
    await expect(
      service.enrollConfirm('user_abc', secret, '000000')
    ).rejects.toThrow();
  });
});

describe('Chiffrement AES-256-GCM', () => {
  it('encrypt() + decrypt() devrait être idempotent', () => {
    const plaintext = 'JBSWY3DPEHPK3PXP'; // secret TOTP exemple
    // @ts-ignore – méthodes privées testées directement
    const encrypted = service['encrypt'](plaintext);
    const decrypted = service['decrypt'](encrypted);
    expect(decrypted).toBe(plaintext);
  });

  it('deux encrypt() du même texte doivent produire des ciphertexts différents (IV aléatoire)', () => {
    // @ts-ignore
    const c1 = service['encrypt']('secret');
    // @ts-ignore
    const c2 = service['encrypt']('secret');
    expect(c1).not.toBe(c2); // IV aléatoire → non déterministe
  });
});
});

```

Extrait 3 — `metrics.service.spec.ts`

TypeScript

```

// src/metrics/metrics.service.spec.ts
describe('MetricsService', () => {
  let service: MetricsService;

  it('devrait incrémenter auth_attempts_total sur authAttempt()', async () => {
    service.authAttempt('login', 'success');
    const metrics = await service.getMetrics();
    expect(metrics).toContain('auth_attempts_total');
    expect(metrics).toContain('method="login"');
    expect(metrics).toContain('result="success"');
  });

  it('devrait enregistrer la latence dans auth_duration_seconds', async () => {
    const end = service.startAuthTimer();
    await new Promise((r) => setTimeout(r, 10)); // 10 ms simulés
    end({ method: 'login' });

    const metrics = await service.getMetrics();
    expect(metrics).toContain('auth_duration_seconds');
  });
});

```

```
});
});
```

Tests composants Frontend (Vitest + RTL — 1 fichier)

`SignalCard.test.tsx`

TypeScript

```
// frontend-web/components/common/SignalCard.test.tsx
import { render, screen } from '@testing-library/react';
import { describe, it, expect } from 'vitest';
import { SignalCard } from '../SignalCard';

const baseSignal = {
  id: 'sig_001', asset: 'BTC', direction: 'BUY', status: 'OPEN',
  entry_price: 67420, stop_loss: 66100, take_profit: 70350,
  confidence: 87.3, patterns: '["bullish_engulfing"]',
  indicators: null, createdAt: new Date().toISOString(),
  exit_price: null, closedAt: null,
};

describe('SignalCard', () => {
  it('devrait afficher l\'asset et la direction', () => {
    render(<SignalCard signal={baseSignal} />);
    expect(screen.getByText('BTC')).toBeInTheDocument();
    expect(screen.getByText(/BUY/i)).toBeInTheDocument();
  });

  it('devrait afficher le score de confiance', () => {
    render(<SignalCard signal={baseSignal} />);
    expect(screen.getByText(/87\.3\s%/) ).toBeInTheDocument();
  });

  it('devrait afficher OPEN en vert pour un signal ouvert', () => {
    render(<SignalCard signal={baseSignal} />);
    const badge = screen.getByText('OPEN');
    // La classe Tailwind indique la couleur green
    expect(badge.className).toMatch(/green/);
  });

  it('devrait afficher exit_price pour un signal CLOSED', () => {
    const closed = { ...baseSignal, status: 'CLOSED', exit_price: 70100,
      closedAt: new Date().toISOString() };
    render(<SignalCard signal={closed} />);
    expect(screen.getByText(/70[\s,.]100/)).toBeInTheDocument();
  });

  // Vérification du design long-only : pas de badge SELL
  it('ne devrait jamais afficher de direction SELL dans le rendu', () => {
    render(<SignalCard signal={baseSignal} />);
    expect(screen.queryByText('SELL')).not.toBeInTheDocument();
  });
});
```

Tests Python (`test\_ai\_module.py`)

Python

```
# ai-module/test_ai_module.py
import unittest
```

```

import numpy as np
import pandas as pd
from indicators_calculator import TechnicalIndicators
from candlestick_patterns import CandlestickPatternDetector
from scoring_engine import ScoringEngine
from dca_simulator import DCASimulator

class TestTechnicalIndicators(unittest.TestCase):
    def setUp(self):
        # Série de prix synthétique pour les tests
        self.close = np.array([100, 102, 98, 105, 110, 108, 112, 115, 111, 118,
                               120, 116, 122, 125, 119, 128, 130, 126, 132, 135],
                               dtype=float)
        self.calc = TechnicalIndicators()

    def test_rsi_range(self):
        """Le RSI doit toujours être compris entre 0 et 100."""
        rsi = self.calc.calculate_rsi(self.close, period=14)
        valid = rsi[~np.isnan(rsi)]
        self.assertTrue(np.all(valid >= 0) and np.all(valid <= 100))

    def test_macd_shape(self):
        """MACD retourne trois arrays de même longueur que l'entrée."""
        macd, signal, hist = self.calc.calculate_macd(self.close)
        self.assertEqual(len(macd), len(self.close))
        self.assertEqual(len(signal), len(self.close))
        self.assertEqual(len(hist), len(self.close))

class TestScoringEngine(unittest.TestCase):
    def test_confidence_bounds(self):
        """Le score de confiance doit toujours être entre 0 et 100."""
        engine = ScoringEngine()
        signals = [
            { 'type': 'indicator', 'strength': 1.5 }, # force > 1 → plafonné
            { 'type': 'candlestick', 'strength': 0.8 },
        ]
        score = engine.compute_confidence(signals)
        self.assertGreaterEqual(score, 0)
        self.assertLessEqual(score, 100)

class TestDCASimulator(unittest.TestCase):
    def test_fixed_mode_deterministic(self):
        """Mode fixed → même résultat à chaque exécution."""
        sim = DCASimulator()
        r1 = sim.simulate(initial=1000, monthly=200, months=12,
                          annual_return=0.12, mode='fixed')
        r2 = sim.simulate(initial=1000, monthly=200, months=12,
                          annual_return=0.12, mode='fixed')
        self.assertEqual(r1['finalBalance'], r2['finalBalance'])

    def test_monte_carlo_non_deterministic(self):
        """Mode monte_carlo (Box-Muller) → résultats différents entre exécutions."""
        sim = DCASimulator()
        r1 = sim.simulate(initial=1000, monthly=200, months=24,
                          annual_return=0.12, volatility=0.05, mode='monte_carlo')
        r2 = sim.simulate(initial=1000, monthly=200, months=24,
                          annual_return=0.12, volatility=0.05, mode='monte_carlo')
        # Avec une volatilité > 0, les deux résultats doivent différer
        self.assertNotEqual(r1['finalBalance'], r2['finalBalance'])

if __name__ == '__main__':
    unittest.main()

```

État de la couverture — bilan honnête

Couche	Fichiers de tests	Ce qui est couvert	Ce qui manque
Backend Auth/MFA	5 fichiers <code>.spec.ts</code>	<code>AuthService</code> , <code>LoggingService</code> , <code>MetricsService</code> , <code>TOTPService</code> , <code>WebAuthnService</code>	<code>StrategiesService</code> , <code>SignalsService</code> , <code>MarketsService</code> , <code>FormationService</code> , <code>SimulatorService</code> , <code>ReportsService</code>
Frontend	1 fichier <code>.test.tsx</code>	<code>SignalCard</code> (rendu, confiance, OPEN/CLOSED)	Pages entières, formulaires, flux auth, graphiques
Python	1 fichier <code>test_ai_module.py</code>	Indicateurs, scoring, DCA	Détection patterns chart/harmoniques/Elliott, bridge stdin/stdout
Tests E2E / intégration HTTP	Aucun	—	Flux complets (register → login → stratégie → signal)

Seuil Jest 70 % : atteint sur les modules testés (auth, MFA) — mais la couverture globale du projet est **inférieure** à ce seuil si l'on compte les services non testés. Le seuil ne porte en pratique que sur les fichiers couverts par les `.spec.ts` existants.

Perspectives d'évolution des tests

Tests d'intégration HTTP — Supertest

TypeScript

```
// Exemple d'extension future : test du flux login complet avec base de données de test
import * as request from 'supertest';
import { INestApplication } from '@nestjsjs/common';

describe('AuthController (e2e)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    // Initialisation de l'application NestJS sur une BDD de test SQLite
    const module = await Test.createTestingModule({ imports: [AppModule] }).compile();
    app = module.createNestApplication();
    await app.init();
  });

  it('POST /auth/register → 201', () => {
    request(app.getHttpServer())
      .post('/auth/register')
      .send({ username: 'test', email: 'test@alvio.io', password: 'P@ssw0rd!' })
      .expect(201)
  });

  afterAll(() => app.close());
});
```

Priorités d'extension

1. Tests d'intégration HTTP (Supertest) sur les flux critiques :

`register` → `login` → `2fa/verify` → `import PDF` → `signal`

2. Tests `StrategiesService` avec mock Anthropic SDK —
vérifier le comportement sur réponse Claude malformée

3. Tests `SignalsService` — déduplication, cycle OPEN→CLOSED,
comportement `EXIT_SIGNAL` sur signal déjà fermé

4. Tests Vitest côté frontend — pages `Login`, `SignalList`,
`StrategyImport` avec MSW (Mock Service Worker) pour simuler l'API

5. Tests Python — bridge stdin/stdout complet avec input OHLCV
de référence et assertion sur le `global_status` retourné

13. Déploiement

Objectifs

La stratégie de déploiement d'Alvio vise trois objectifs :

- 1. Reproductibilité** — l'environnement de développement local et l'environnement de production doivent être identiques à la stack technique près. Docker Compose garantit cette parité.
- 2. Isolation des services** — chaque composant (base de données, cache, API, monitoring) tourne dans son propre conteneur avec des dépendances de démarrage déclarées.
- 3. Observabilité intégrée** — la stack de monitoring (Prometheus, Grafana, ELK) est embarquée dans le même `docker-compose.yml` et démarre en même temps que l'application.

Conteneurisation

Vue d'ensemble — 9 services Docker

Le fichier `docker-compose.yml` (racine du projet) orchestre **9 services** :

Service	Image	Port exposé	Rôle
postgres	postgres:15-alpine	5432	Base de données principale (11 modèles Prisma)
redis	redis:7-alpine	6379	Cache, sessions JWT, OTP, compteurs de verrouillage
backend	Build local	3001	API NestJS — cœur applicatif
frontend	Build local	3000	Application Next.js 13
prometheus	prom/prometheus:v2.51.0	9090	Collecte des métriques backend
grafana	grafana/grafana:10.4.0	3003	Dashboards (17 panneaux sécurité)
elasticsearch	elasticsearch:8.13.0	9200	Moteur SIEM — stockage des logs
kibana	kibana:8.13.0	5601	Interface SIEM + 5 règles de détection
logstash	logstash:8.13.0	514/udp	Pipeline Syslog UDP → Elasticsearch

Démarrage complet en une commande :

Bash / Shell

```
docker-compose up -d
# ou pour l'environnement de dev (sans monitoring lourd) :
```

Dockerfile Backend — `backend-code/Dockerfile`

Dockerfile

Points notables :

- `npx prisma generate` s'exécute au build — le client TypeScript typé est intégré à l'image, pas généré à chaud au démarrage.
- `prisma db push` au démarrage du conteneur applique le schéma sans migration versionnée (adapté au développement ; en production, `prisma migrate deploy` serait préférable).
- Le volume `./backend-code/ai-module:/app/ai-module` dans `docker-compose.yml` monte le module Python depuis l'hôte — le script `pattern_detector.py` est donc mis à jour sans rebuild de l'image.

Dockerfile Frontend — `frontend-web/Dockerfile`

Dockerfile

Points notables :

- **Multi-stage build** : l'image finale ne contient pas le code source TypeScript ni les devDependencies — taille réduite et surface d'attaque limitée.
- **node:22-alpine** pour le frontend vs **node:18-alpine** pour le backend — écart volontaire lié aux contraintes de compatibilité de certaines dépendances NestJS sur Node 22.

Extrait docker-compose.yml — services critiques

YAML

```

DATABASE_URL:      postgresql://user:pass@postgres:5432/brokeria
REDIS_URL:         redis://redis:6379
JWT_SECRET:        ${JWT_SECRET:?JWT_SECRET is required}          # obligatoire
JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET:?JWT_REFRESH_SECRET is required}
TOTP_ENCRYPTION_KEY: ${TOTP_ENCRYPTION_KEY:?TOTP_ENCRYPTION_KEY is required}
ANTHROPIC_API_KEY:  ${ANTHROPIC_API_KEY}
SYSLOG_HOST:       logstash          # logs → Logstash dans le même réseau Docker
SYSLOG_PORT:       514
depends_on:
  postgres: { condition: service_healthy }  # attend que PG soit prêt
  redis:    { condition: service_healthy }
volumes:
  - ./backend-code/ai-module:/app/ai-module # module Python monté en volume

logstash:
  image: docker.elastic.co/logstash/logstash:8.13.0
  ports:
    - "514:514/udp"      # Syslog UDP depuis le backend NestJS → Logstash
  depends_on:
    elasticsearch: { condition: service_healthy }
```

Ordre de démarrage garanti par les `depends\_on`



CI/CD — État réel

Aucun pipeline CI/CD automatisé n'est configuré dans le projet.

La vérification via Glob confirme l'absence de workflows GitHub Actions dans le répertoire `.github/workflows/` du projet (les fichiers `.yaml` présents appartiennent exclusivement aux `node_modules` de dépendances tierces).

Le cycle de livraison actuel est entièrement **manuel** :

- 1. Développement local → `npm run start:dev`
- 2. Tests manuels → Postman / `curl`
- 3. `npm run test` (Jest — seuil 70 %)
- 4. Vérification TypeScript → `npm run build` (0 erreur)
- 5. Test Docker local → `docker-compose up --build`
- 6. Commit + push Git

Cibles de déploiement — Statut

Ce qui est implémenté

Élément	Statut	Détails
<code>docker-compose.yaml</code>	Implémenté	9 services, healthchecks, volumes persistants

Élément	Statut	Détails
backend-code/Dockerfile	Implémenté	node:18-alpine, prisma generate, multi-stage absent
frontend-web/Dockerfile	Implémenté	node:22-alpine, multi-stage build
Module Python dans Docker	Implémenté	Monté en volume /app/ai-module
Variables d'env documentées	Implémenté	Toutes les variables déclarées dans docker-compose.yml

Ce qui est prévu (non encore effectué)

Élément	Statut	Raison
Déploiement Railway (backend + DB + Redis)	Prévu	Le Dockerfile backend contient un commentaire Railway (Railway injecte PORT) mais aucun railway.toml n'existe
Déploiement Vercel (frontend)	Prévu	Aucun vercel.json configuré — compatible par défaut avec Next.js mais non déployé
GitHub Actions CI/CD	Prévu	Aucun workflow .github/workflows/ dans le projet
prisma migrate deploy en prod	Prévu	La commande actuelle utilise prisma db push — adaptée au dev, non recommandée en prod

Variables d'environnement de production

Toutes les variables marquées :? dans le docker-compose.yml sont obligatoires — Docker refuse de démarrer si elles sont absentes.

Variable	Obligatoire	Description
JWT_SECRET	Oui	Secret HMAC-SHA256 pour les access tokens
JWT_REFRESH_SECRET	Oui	Secret pour les refresh tokens (JTI rotation)
TOTP_ENCRYPTION_KEY	Oui	64 caractères hex (32 bytes) — AES-256-GCM des secrets TOTP
DATABASE_URL	Oui	URL PostgreSQL complète
REDIS_URL	Oui	URL Redis
ANTHROPIC_API_KEY	Non (opt.)	Clé Claude API — AIService désactivé si absent
SMTP_HOST / SMTP_USER / SMTP_PASS	Non (opt.)	Email SMTP — OTP/MagicLink désactivés si absents
GITHUB_CLIENT_ID / GITHUB_CLIENT_SECRET	Non (opt.)	OAuth GitHub — route désactivée si absents

Variable	Obligatoire	Description
COINGECKO_API_KEY	Non (opt.)	Clé CoinGecko — fonctionne sans (tier gratuit sans header)
FRONTEND_URL	Non (opt.)	Default http://localhost:3002 — CORS origin
GRAFANA_PASSWORD	Non (opt.)	Default brokeria_grafana (à changer en prod)

Perspectives CI/CD

La mise en place d'un pipeline automatisé est la principale évolution de la chaîne de livraison. Un workflow GitHub Actions minimal couvrirait :

YAML

```
# .github/workflows/ci.yml — exemple cible (non implémenté)
name: CI

on: [push, pull_request]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:15, env: { POSTGRES_PASSWORD: test } }
      redis: { image: redis:7 }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18' }
      - run: npm ci
        working-directory: backend-code
      - run: npx prisma generate && npm run test:cov
        working-directory: backend-code

  build-docker:
    needs: test-backend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t alvio-backend ./backend-code
      - run: docker build -t alvio-frontend ./frontend-web

  deploy-railway:
    needs: build-docker
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: bervProject/railway-deploy@main
        with: { railway_token: ${ secrets.RAILWAY_TOKEN }, service: 'backend' }
```

Les étapes prioritaires pour atteindre une livraison continue sont :

1. Créer [railway.toml](#) (backend) et déployer sur Railway
2. Connecter le repo GitHub à Vercel (frontend — déploiement automatique sur merge)
3. Ajouter le workflow GitHub Actions (tests + build + deploy)
4. Remplacer [prisma db push](#) par [prisma migrate deploy](#) pour les

migrations versionnées et réversibles en production

14. Annexes

A. Diagrammes

Les diagrammes ci-dessous ont été générés via le script `/docs/diagrams/generate_all.py` (matplotlib 3.10.8, palette Alvio) à partir des données réelles du projet (schema.prisma, controllers, services).

A.1 — MCD — Modèle Conceptuel de Données

Fichier : `docs/img/mcd.png` (voir page X)

Représente les 11 entités du domaine Alvio et leurs associations sémantiques : User, Signal, Strategy, Report, AuthLog, WebAuthnCredential, SimulationResult, PortfolioSnapshot, Course, Lesson, UserProgress. Met en évidence le design long-only (Signal.direction) et la nullable de AuthLog.userId.

A.2 — MLD — Modèle Logique de Données

Fichier : `docs/img/mld.png` (voir page X)

Traduction relationnelle du MCD : clés primaires CUID, clés étrangères avec actions référentielles (CASCADE / SetNull), contraintes UNIQUE composites (`[userId, month, year]` sur Report et PortfolioSnapshot ; `[userId, lessonId]` sur UserProgress).

A.3 — MPD — Modèle Physique de Données

Fichier : `docs/img/mpd.png` (voir page X)

Types SQL PostgreSQL 15 (`CHAR(25)`, `FLOAT8`, `SMALLINT`, `TIMESTAMP TZ`, `TEXT`), valeurs par défaut, index déclarés. Index composite critique sur Signal : `[strategyId, asset, direction, status]` — utilisé par la déduplication des signaux BUY OPEN.

A.4 — Diagramme de Classes NestJS

Fichier : `docs/img/classes.png` (voir page X)

Architecture en 4 couches du backend Alvio : Controllers (14), Services métier (AuthService, StrategiesService, SignalsService, MarketsService...), Services infrastructure (AIService Claude, PatternDetectionService spawn, TOTPSERVICE, WebAuthnService, LoggingService, MetricsService), Couche données (PrismaService, RedisService, EmailService, APIs externes).

A.5 — Diagramme de Cas d'Utilisation

Fichier : `docs/img/use_cases.png` (voir page X)

4 acteurs (Trader, Claude API, CoinGecko, SMTP) et 17 cas d'utilisation couvrant l'authentification MFA (6 UC), les marchés (2 UC), les modules IA (4 UC), la formation (2 UC) et les outils (3 UC). Frontière système délimitée.

A.6 — Séquence — Création de Stratégie IA

Fichier : [docs/img/seq_strategy.png](#) (voir page X)

Flux en 14 étapes : upload PDF → Multer memoryStorage → pdf-parse v2 `PDFParse.getText()` (tronqué 15 000 chars) → Claude API `claude-sonnet-4-6` → `cleanJsonResponse()` + `validateStrategyRules()` → `Prisma.strategy.create()` → spawn `python3 pattern_detector.py` (stdin/stdout) → Signal BUY si `ENTRY_SIGNAL`.

A.7 — Séquence — Authentification Multi-Facteurs

Fichier : [docs/img/seq_auth.png](#) (voir page X)

3 phases successives : (1) email/password + `bcrypt.compare` + verrouillage Redis après 3 échecs (`locked:{userId}`) ; (2) OTP email via `preauth:{token}` Redis TTL 600 s ; (3) émission JWT avec rotation JTI (`refresh:{jti}` Redis 7 j + cookie `httpOnly SameSite:strict`).

A.8 — Architecture Globale

Fichier : [docs/img/architecture.png](#) (voir page X)

Vue d'ensemble de l'écosystème Alvio : clients Next.js/React Native → NestJS :3001 (middleware Helmet/CORS, 14 controllers, guards JWT) → PostgreSQL 15 / Redis 7 / Python spawn stdin/stdout → Claude API + CoinGecko REST → stack monitoring (Prometheus :9090 + Grafana :3003 + ELK :9200/:5601/:514/udp).

B. Extraits de code complets

B.1 — SYSTEM_PROMPT complet — Claude Strategy Engine

Prompt système envoyé à `claude-sonnet-4-6` dans `AIService.analyzeStrategyDocument()`. Définit le contrat de sortie JSON strict.

```
// src/ai/ai.service.ts — SYSTEM_PROMPT (variable constante)

Tu es un expert en analyse technique et en trading algorithmique.
Ton rôle est d'analyser les documents de stratégie de trading et d'en extraire
les règles de manière structurée.

IMPORTANT : Réponds UNIQUEMENT avec un objet JSON valide, sans texte supplémentaire,
sans balises markdown, sans explication. Structure exacte requise :

{
  "name": "string — nom court de la stratégie",
  "description": "string — résumé en 1-2 phrases",
  "asset": "string — actif principal (ex: BTC, ETH, SOL)",
  "timeframe": "string — timeframe principal (15m | 1h | 4h | 1d)",
  "entry_conditions": [
    "string — condition d'entrée 1 (ex: RSI < 30)",
    "string — condition d'entrée 2 (ex: EMA20 > EMA50)"
  ],
  "exit_conditions": [
    "string — condition de sortie 1 (ex: RSI > 70)",
    "string — condition de sortie 2 (ex: prix atteint take_profit)"
  ],
  "indicators": [
    { "name": "string — nom de l'indicateur", "params": { "period": number } }
  ],
  "risk_management": {
    "stop_loss_percent": number,
```

B.2 — Format JSON complet échangé via le bridge Python

Entrée envoyée par **PatternDetectionService** sur stdin, et sortie reçue sur stdout.

JSON

B.3 — Variables d'environnement complètes (template `.env` backend)`

Bash / Shell

B.4 — Commandes de démarrage — référence rapide

Bash / Shell

15. Mapping des Blocs de Compétences RNCP 37873 — CDA

Titre : Concepteur Développeur d'Applications — RNCP 37873 — Niveau 6

Candidat : Adam Ijjai — Promotion 2025-2026

Projet : Alvio — Plateforme de trading propulsée par IA

Ce tableau de correspondance relie chaque compétence du référentiel RNCP 37873 à une section du présent dossier et à un élément technique réel du projet Alvio. Les compétences partiellement couvertes sont signalées honnêtement.

Bloc 1 — Développer une application sécurisée

CP1 — Installer et configurer son environnement de travail en fonction du projet

Compétence	Preuves dans ce dossier	Éléments techniques réels
Choisir et installer les outils de développement adaptés au projet	§04 Gestion de projet	Node.js 18, Python 3, Expo CLI, Prisma CLI, Docker Desktop
Configurer un environnement de développement local reproductible	§13 Déploiement	<code>docker-compose.yml</code> — <code>docker-compose up -d postgres redis</code> lance l'environnement complet en une commande
Gérer les versions avec un SCM	§04 Gestion de projet	Git mono-repo, commits conventionnels (<code>feat:</code> , <code>fix:</code> , <code>refactor:</code>)
Configurer les variables d'environnement sensibles	§08 Architecture, §13 Déploiement	<code>.env</code> backend (DATABASE_URL, JWT_SECRET, ANTHROPIC_API_KEY, TOTP_ENCRYPTION_KEY) — variables obligatoires marquées <code>?</code> dans <code>docker-compose.yml</code>

CP2 — Développer des interfaces utilisateur

Compétence	Preuves dans ce dossier	Éléments techniques réels
Développer des interfaces web responsive	§05 UI/UX, §09 Application Web	Next.js 13 Pages Router, Tailwind CSS, 20+ pages (dashboard, markets, signaux, formation, simulateur, MFA)
Intégrer une charte graphique cohérente	§05 UI/UX	Palette Alvio (amber #f59e0b, vert #10b981, navy #0f172a, violet #6366f1), glassmorphism, framer-motion
Développer une interface mobile native	§10 Application Mobile	React Native 0.72.3 / Expo 49 — 5 écrans (Login, Dashboard, Signals, Simulator, Profile), navigation Stack + Bottom Tabs

Compétence	Preuves dans ce dossier	Éléments techniques réels
Afficher des données graphiques interactives	§09 Application Web	<code>lightweight-charts 4.2.3</code> (TradingView) — graphiques chandeliers OHLCV avec fallback line chart
Respecter les principes d'accessibilité	§05 UI/UX	Contraste > 7:1 (WCAG AA), aria-labels sur icônes, navigation clavier

CP3 — Développer des composants métier

Compétence	Preuves dans ce dossier	Éléments techniques réels
Développer la logique métier côté serveur	§08 Architecture, §11 Module IA	NestJS 10 — services modulaires (AuthService 519L, StrategiesService, SignalsService, MarketsService...)
Intégrer une API LLM pour le traitement du langage	§11 Module IA	<code>AIService.analyzeStrategyDocument()</code> → Claude API <code>claude-sonnet-4-6</code> , max_tokens 4096, SYSTEM_PROMPT JSON strict
Développer un bridge inter-processus vers un module Python	§11 Module IA	<code>PatternDetectionService</code> → <code>spawn('python3', ['pattern_detector.py'], { stdio: ['pipe', 'pipe', 'pipe'] })</code> — entrée/sortie JSON via stdin/stdout
Implémenter des algorithmes de trading et d'analyse technique	§11 Module IA	15 scripts Python (RSI, MACD, Bollinger, ATR, Ichimoku, Elliott, harmoniques Gartley/Butterfly/Bat/Crab, scoring engine)
Développer un simulateur financier	§09 App Web, §10 App Mobile, §11 IA	<code>SimulatorService</code> — DCA mode <code>fixed</code> (déterministe) et <code>monte_carlo</code> (Box-Muller Gaussian)
Implémenter un système de signaux de trading	§11 Module IA	<code>SignalsService</code> — design long-only, déduplication, cycle OPEN→CLOSED, scheduler <code>@Cron('/15 *')</code>
Développer un module de formation (LMS)	§09 Application Web	<code>FormationService</code> — Course/Lesson/UserProgress, types VIDEO/ARTICLE/QUIZ, progression par utilisateur
Générer des rapports calculés à la demande	§11 Module IA	<code>ReportsService.getMonthlyStats()</code> — win rate, P&L estimé, patterns détectés, indicateurs utilisés

CP4 — Contribuer à la gestion d'un projet informatique

Compétence	Preuves dans ce dossier	Éléments techniques réels
Planifier et découper un projet en itérations	§04 Gestion de projet	Scrum solo — 10 sprints documentés (S1 Fondations → S10 Finalisation)
Identifier et gérer les risques projet	§04 Gestion de projet	7 risques documentés (429 CoinGecko, réponse Claude malformée, TA-Lib, dérive de périmètre...) avec mitigation effective

Compétence	Preuves dans ce dossier	Éléments techniques réels
Produire de la documentation technique	§01-§16 (ce dossier)	DONNEES_REELLES.md, PLAN.md, sections /docs/sections/ , diagrammes /docs/img/
Gérer le backlog et les priorités en solo	§04 Gestion de projet	PBS complet (5 branches), backlog figé après S4, features reportées explicitement en perspectives

Bloc 2 — Concevoir et développer une application sécurisée organisée en couches

CP5 — Analyser les besoins et maquetter une application

Compétence	Preuves dans ce dossier	Éléments techniques réels
Identifier les acteurs et les cas d'utilisation	§07 UML	Diagramme use_cases.png — 4 acteurs (Trader, Claude API, CoinGecko, SMTP), 17 cas d'utilisation
Produire des zonings et wireframes	§05 UI/UX	Zonings ASCII pour Dashboard, page Marchés, Import PDF, 2 écrans mobiles (Dashboard, Simulator)
Définir une charte graphique	§05 UI/UX	Palette 6 couleurs, glassmorphism, framer-motion, iconcons
Formaliser la problématique et les livrables	§03 Introduction	Problématique en 3 défis, tableau des 5 livrables avec versions réelles, vue d'ensemble ASCII

CP6 — Définir l'architecture logicielle d'une application

Compétence	Preuves dans ce dossier	Éléments techniques réels
Concevoir une architecture en couches	§08 Architecture, §07 UML	4 couches NestJS documentées (Controllers → Services → Infra → Data) ; classes.png
Choisir et justifier les patterns architecturaux	§08 Architecture	IoC/DI NestJS, modules isolés (@Module), services globaux (@Global()), ValidationPipe whitelist
Concevoir le flux de données entre les composants	§07 UML, §11 IA	Diagrammes de séquence seq_strategy.png (PDF→Claude→Signal) et seq_auth.png (MFA 3 phases)
Sécuriser les échanges entre couches	§08 Architecture	JWT Bearer en header (pas URL), cookie httpOnly SameSite:strict , Prisma requêtes paramétrées, Helmet
Concevoir l'architecture globale multi-couches	§08 Architecture	architecture.png — Next.js → NestJS → PostgreSQL/Redis + Python spawn + Claude API + CoinGecko

CP7 — Concevoir et mettre en place une base de données relationnelle

Compétence	Preuves dans ce dossier	Éléments techniques réels
Produire un dictionnaire de données	\$06 BDD	11 tables documentées — champs, types SQL réels, contraintes, valeurs par défaut
Modéliser le MCD	\$06 BDD	mcd.png — 11 entités, cardinalités réelles, choix de design (AuthLog.userId nullable, long-only Signal)
Produire le MLD	\$06 BDD	mld.png — PK CUID, FK avec CASCADE/SetNull, contraintes UNIQUE (userId+month+year, userId+lessonId)
Produire le MPD	\$06 BDD	mpd.png — types PostgreSQL 15, index composites ([strategyId, asset, direction, status]), FLOAT8, SMALLINT
Justifier les choix de conception	\$06 BDD	CUID vs entiers séquentiels (sécurité), JSON en TEXT (compat. SQLite dev), SetNull vs CASCADE pour UserProgress

CP8 — Développer des composants d'accès aux données SQL et NoSQL

Compétence	Preuves dans ce dossier	Éléments techniques réels
Implémenter des requêtes SQL via un ORM	\$08 Architecture, \$11 IA	Prisma v5 — findMany, findFirst, create, update, upsert — requêtes paramétrées (anti-SQLi natif)
Utiliser des index pour optimiser les requêtes	\$06 BDD	Index [userId], [asset], [strategyId, asset, direction, status] — déduplication en O(log n)
Implémenter un cache Redis (NoSQL clé-valeur)	\$08 Architecture, \$09 App Web	RedisService (ioredis) — double clé fraîche + stale (anti-429), sessions JWT (refresh:{jti}), verrouillage (locked:{userId})
Gérer les migrations de schéma	\$13 Déploiement	prisma db push (dev) — prisma migrate deploy identifié comme amélioration prod
Implémenter des contraintes d'intégrité	\$06 BDD	@unique, @default, FK avec actions référentielles, @@unique([userId, month, year])

Bloc 3 — Préparer le déploiement d'une application sécurisée

CP9 — Préparer et exécuter les plans de tests d'une application

Compétence	Preuves dans ce dossier	Éléments techniques réels
Rédiger des tests unitaires pour les services critiques	\$12 Tests	5 fichiers Jest — AuthService (login/lockout/2FA/PIN), TOTPService (AES-256-GCM), LoggingService, MetricsService, WebAuthnService

Compétence	Preuves dans ce dossier	Éléments techniques réels
Tester les composants UI	§12 Tests	Vitest + @testing-library/react — <code>SignalCard.test.tsx</code> (BUY/CLOSED/confidence/long-only)
Tester les modules Python	§12 Tests	<code>test_ai_module.py</code> — RSI bornes, DCA fixed déterministe vs monte_carlo stochastique
Définir un seuil de couverture	§12 Tests	Jest <code>coverageThreshold: 70 %</code> (branches, functions, lines, statements)
Tests d'intégration HTTP (Supertest)	<i>Partiellement couvert</i>	Non implémentés dans la version actuelle — présentés en perspectives (§12), flux validés manuellement via Postman/cURL

CP10 — Préparer et documenter le déploiement d'une application

Compétence	Preuves dans ce dossier	Éléments techniques réels
Rédiger les Dockerfiles de l'application	§13 Déploiement	<code>backend-code/Dockerfile</code> (node:18-alpine, prisma generate, python3+pandas+numpy) et <code>frontend-web/Dockerfile</code> (multi-stage node:22-alpine)
Orchestrer les services avec Docker Compose	§13 Déploiement	<code>docker-compose.yml</code> — 9 services, healthchecks, volumes persistants, dépendances déclarées
Documenter les variables d'environnement	§13 Déploiement	Tableau complet des variables obligatoires/optionnelles, valeurs par défaut, descriptions
Documenter l'architecture de déploiement	§08 Architecture, §13 Déploiement	Vue d'ensemble schéma ASCII, ports exposés, réseau Docker interne (backend→logstash via SYSLOG_HOST)
Déploiement effectif sur une plateforme cloud	<i>Prévu</i>	Non encore déployé — Railway (backend) et Vercel (frontend) identifiés comme cibles, Dockerfiles compatibles, mais <code>railway.toml</code> et <code>vercel.json</code> absents

CP11 — Contribuer à la mise en production dans une démarche DevOps

Compétence	Preuves dans ce dossier	Éléments techniques réels
Mettre en place une observabilité applicative	§08 Architecture, §13 Déploiement	Prometheus <code>prom/prometheus:v2.51.0</code> — scrape <code>/metrics</code> toutes les 15 s, 4 compteurs + 1 histogram + 1 gauge
Mettre en place des dashboards de métriques	§13 Déploiement	Grafana <code>grafana/grafana:10.4.0</code> — 17 panneaux (authentification, MFA, latence, sessions actives)
Implémenter un pipeline de logs centralisé	§08 Architecture, §13 Déploiement	RFC 5424 Syslog UDP → Logstash → Elasticsearch 8.13.0 — index <code>brokeria-security-YYYY.MM.DD</code>

Compétence	Preuves dans ce dossier	Éléments techniques réels
Définir des règles de détection SIEM	§13 Déploiement	Kibana — 5 règles de détection (brute-force, compte verrouillé, IP suspecte, anomalie auth, jeu)
Implémenter un audit trail d'authentification	§08 Architecture	<code>LoggingService</code> → <code>Prisma.authLog.create()</code> + Syslog UDP — chaque tentative tracée avec <code>userId</code> , action, résultat, IP
Pipeline CI/CD automatisé	Prévu	Non implémenté — aucun workflow <code>.github/workflows/</code> dans le projet ; cycle de livraison manuel documenté en §13 ; workflow GitHub Actions cible décrit en perspectives

Synthèse par bloc

Bloc	Compétences	Pleinement démontrées	Partiellement	Non couvertes
Bloc 1 — Développement sécurisé	16	14	0	2 (<code>backtestStrategy</code> , <code>getPatternsHistory</code> non fonctionnels)
Bloc 2 — Conception en couches	15	15	0	0
Bloc 3 — Déploiement	11	8	3	0

Les 3 compétences partiellement couvertes du Bloc 3 (tests d'intégration HTTP, déploiement cloud effectif, CI/CD automatisé) sont documentées en tant que **perspectives d'évolution** dans les sections correspondantes du dossier. Elles ne sont pas revendiquées comme réalisées.

Points forts de la démonstration

Pour le jury, les éléments les plus significatifs au regard du niveau 6 (Bac+4) sont les suivants :

Élément	Niveau de complexité	Référence
Système MFA complet from scratch (OTP, TOTP AES-256-GCM, WebAuthn FIDO2, PIN, Magic Link, GitHub OAuth)	Élevé	§08, §07 seq_auth.png
Bridge NestJS→Python via <code>spawn</code> stdin/stdout avec données OHLCV	Élevé	§11, <code>pattern-detection.service.ts</code>
Intégration Claude API avec prompt engineering et validation de réponse JSON	Élevé	§11, <code>ai.service.ts</code>
Architecture en 4 couches NestJS avec 14 controllers, 40+ endpoints, modules isolés	Élevé	§08, classes.png

Élément	Niveau de complexité	Référence
Stack monitoring complète (Prometheus + Grafana 17 panneaux + ELK + 5 règles SIEM)	Élevé	\$13, docker-compose.yml
BDD PostgreSQL 11 modèles conçus depuis le MCD jusqu'au MPD avec index optimisés	Moyen-élevé	\$06, mcd/mld/mpd.png
Cache Redis anti-429 double clé (TTL frais + stale 1h)	Moyen	\$08, markets.service.ts
Application mobile React Native / Expo avec 5 écrans fonctionnels	Moyen	\$10